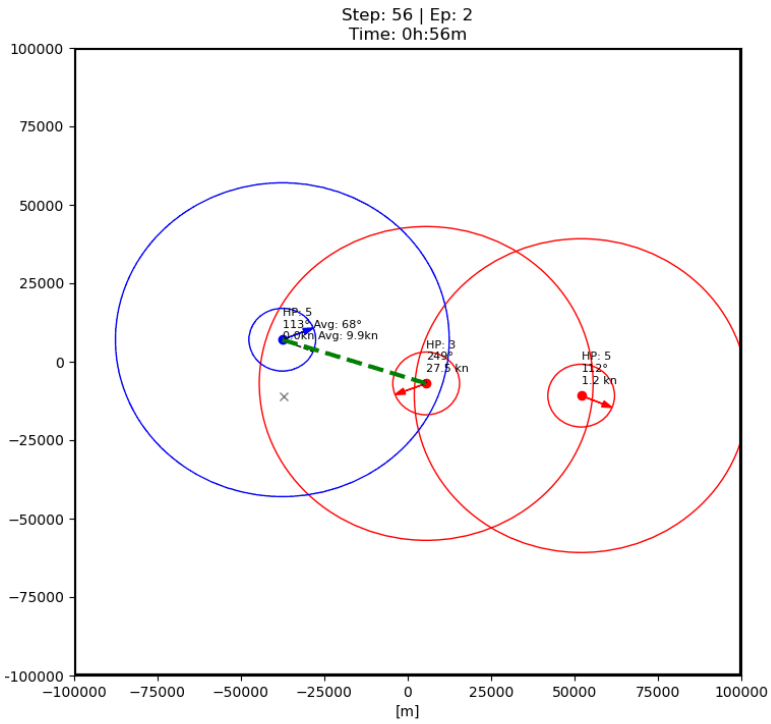
UNIVERSITY OF COPENHAGEN
FACULTY OF SCIENCE



Master Thesis

Boosted Decision-Making with Adversarial Reinforcement Learning Agents in Naval Warfare

Michael Bach

University Advisor: Professor Troels Christian Petersen
Company Advisor: Jens Egeløkke Frederiksen, Terma

Submitted: May 20, 2025

This thesis has been submitted to The Faculty of Science, University of Copenhagen

Chapter 1

Abstract

In this thesis, I attempt to develop a method for boosting decision-making in naval warfare. By analyzing chess tactics and the use of chess engines by professional players the recent decades, I lay a mathematical foundation for military tactics through the lens of game theory and discuss how these tactics can augment military expertise and boost decision-making. The method attempted is a model of adversarial deep deterministic policy gradient agents that, through reward shaping, optimize tactics by searching through the state-space of a naval simulator with noise exploration and training competitively against each other. By taking inspiration from nature, emergent tactics are encouraged by modeling rewards as energy potential fields and in similar fashion to energy minimization problems, make the agents seek trajectories in the field which maximize reward. As a case study, the model is then applied to a simple naval simulator capturing the essence of anti-surface warfare in the maritime domain. Results from the optimization process show deep problems with the training evolution of the agents, indicating that the approach is flawed as suboptimal tactics emerge.

Contents

1 Abstract **3**

Contents **4**

2 Forewords **7**

3 Introduction **9**
- Problem Statement . . . 17
- Research Questions . . . 17
- Approach and Scope . . . 17
- Thesis Structure . . . 18

4 Literature Review **19**
- Reinforcement Learning Fundamentals . . . 20
- Adversarial Approaches in Reinforcement Learning . . . 21
- Applications to Military Context . . . 22

5 The Mathematics of Tactics **26**
- The Game Tree of Chess . . . 27
- Chess Tactics . . . 29
- A Game of Warships . . . 34
- Tactical Categories . . . 38
- Optimization in Hybrid Spaces . . . 40

6 Synthetic Commanders **43**
- The Bellman Equation . . . 44
- The Deep Deterministic Policy Gradient Algorithm . . . 45
- Adversarial Commanders . . . 50
- Reward Shaping with Fields . . . 52
- Hyperparameter Setting . . . 58
- Stability Enhancements . . . 64
- Environment Compatibility . . . 64

7 Results **67**

Simple Attraction 68
Cat and Mouse 69
Distinguishing Separate Tactics 70

8 Discussion **73**
Suboptimal Emergent Policies 74
Potential Future Applications 74

9 Conclusion **77**

10 Challenges **79**
Attempts at Dealing with Hybrid State-spaces 79

11 Future Research **83**

Bibliography **85**
Appendix A: Machine Learning and Neural Networks 89
A.1 Unsupervised Learning 89
A.2 Supervised Learning 89
A.3 The Perceptron 90
A.4 Neural Networks 90
A.5 Backpropagation 91

Chapter 2

Forewords

At the age of five, my grandfather taught me the rules of chess. Whenever I visited, we would play and he would, without exception, beat me relentlessly and without the slightest hint of mercy. Taking his time after each game to explain to me every erroneous decision I had made and how he had exploited it. He was the headmaster of the local vocational college and had risked his own life and his family's lives helping the Danish resistance movement during the German occupation and as such was a fierce and tough opponent. Our games continued until three months before his death, a bit before his health began to deteriorate. I finally managed to pin down his queen and check-mated him shortly after, allowing me a win after eight straight years of defeat. That game remains one of the proudest moments of my life and an ending fit for the great relationship I had with him.

I dedicate this work to you,

Svend Ove Bach

Thank you for teaching me the virtue of defeat and the value of consistency.

Your grandson, Michael Bach

Chapter 3

Introduction

NAVAL warfare has historically had a significant role in how states have developed economically, politically, and socially. From the early settlers of ancient history and through to the modern era, control of the sea - also known as *Sea Power* - has allowed nations to project power and influence to lands far away from their physical borders. This capacity to traverse the world's oceans and conduct far-reaching operations has not only induced the expansion of trade routes but has allowed military force to be rapidly deployed, allowing states to affect the course of conflicts and the evolution of empires.

The shape of warfare through its structure, methods, organization, and technology has changed radically with technological and social development over history, yet the strategic goal has remained the same, providing maritime security by dominating the maritime space to secure state interests. Summarized well by the slogan of the Royal Danish Navy, but certainly applicable to all naval states;

The Royal Danish Navy defends Danish interests at Sea - With power if necessary[1]

A Brief History of Naval Tactics

Looking back at ancient history, some of the earliest records of naval warfare were in the Mediterranean world. Early small states and later Rome took advantage of naval supremacy and used their Sea Power to not only defend their own local sea territory, but also to disturb enemy trade and control supply lines. Warships at the time were rowed galley ships, and the actual combat was carried out with literal hand-to-hand fighting when two ships grappled each other. Despite the relative simplicity of this early sea combat compared to modern warships, the role of the early sea soldiers required an extensive training regiment, complex logistical networks and leaders employing

[1] Translated from Navy [2025], originally "Søværnet forsvarer Danmarks interesser fra havet – med magt om nødvendigt."

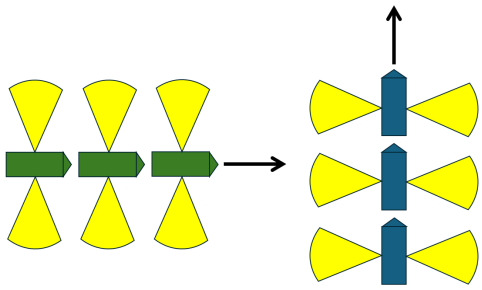


Figure 3.1: Illustration of the "Crossing the T" naval tactic employed in the age of sail. Ships of the line in formation 1 divided in green and blue fleets, with cannon firearchs in yellow. Blue fleet is able to put all their entire port side broadside to bear on the green fleet, without them being able to return the fire. Which ever commander that could outmaneuver the opponent in the above constellation would have a massive advantage.

decisive and smart tactics to exploit environmental factors like wind direction, fogs, and water currents.

Naval warfare later developed into wind powered ships, signifying the beginning of the *Age of Sail*. Large fleets of massive warships with huge arrays of iron cannons lined up on the oceans, in gigantic engagements that would decide the fate of nations. This also marked a paradigm shift in naval tactics from close combat to advanced ship maneuvering in order to maximize the effectiveness of cannons against an opponent. The goal of commanders was now to maneuver around the enemy and keep them within range of their side cannons, while avoiding that the opponent does the same. The classical "Crossing the T" was at the time widely accepted as the best tactic optimizing these basic principles. Fleets of warships sailing in *formation 1*, a column line of ships following each others wake, tried to cross ahead of the fleet in order to maximize use of their side cannons, famously "Crossing the T" of the opponent - See figure 3.1. These tactics also required new advances to fleet management, communication and not least new forms of leadership exemplified by legendary admirals of the time like Peder Wessel Tordenskiold or Lord Horatio Nelson.

The age of sail began its conclusion when the industrial revolution swept across the world. Steam propulsion, steel hulls, and massive artillery pieces emerged and vastly outperformed the sailing vessels of the old world. Throughout the twentieth century, this technological development accelerated and resulted in massive dreadnoughts, aircraft carriers, and nuclear submarines. The modern naval battlefield is dominated by multi-domain integrated combat systems, various stealth technologies and computer guided weapon systems, all controlled and manned by specialists handling the ever-growing list of advanced new equipment. The arrival of the digital age at the beginning of the

twenty-first century marked the start of another new paradigm in the way naval warfare was conducted. Sensors, weapons and communication are all largely digital and connect to one central combat management system, often linked together fleet-wide across both air and naval units. It is in this central combat management system that warfare officers are presented with the real-world data, conduct their decision-making and promulgate their courses of action.

One of the first to try to systematically describe the basic strategic principles of naval warfare and its historical impact was the American naval officer Alfred Thayer Mahan in his famous book, *The Influence of Sea Power upon History, 1660-1783*, Mahan [1890]. Scouring through the history books of naval conflicts and maritime trade patterns, he was among the first to academically conclude that mastery of the seas had been an important catalyst of growing state power. Many of his principles has stood the test of time, albeit some refined over the years, and have proven useful tools for teaching the new generations of warfare officers. Principles like the importance of concentrating naval forces, control of maritime choke points and the role of commerce protection remain ever relevant.

Underlying this work and its historic relevance is the importance of navies to teach their naval officers in making critical tactical decisions with wide-reaching operational or strategic consequences under conditions of stress and uncertainty. Increasingly in resent time, this decision-making process uses advanced technology to gain and maintain situational awareness, interpret threat, risk- assess and mitigate, and coordinate the actions among naval units.

The Bridge between Games and Warfare

An famous old Greek quote of unknown author, often attributed to Aristotle or Anacharsis, exemplify the dread the oceans have represented for mariners throughout the ages;

There are three sorts of people:
those that are alive,
those that are dead,
and those who are at sea

Simply put, those who went to sea were both dead and alive until they returned. Much like Schrödingers Cat in quantum mechanics, though in a much less physical sense, lack of information about the subjects caused them to be interpreted as both.

Naval warfare is by its nature unintuitive for those without maritime experience. Historically, only a small part of the population has had the privilege of experiencing life at sea, and those that did quickly learned that it is nothing like land-based living. The platform is ever-moving and the weather impact

is world-defining, the living quarters are tight and privacy a limited resource, emergencies had to be dealt with locally and rapid technological change required quick adaptation and resilience from seafarers.

This sets the demand for a highly specialized skill set for navy soldiers. A skill set that has changed through the ages to reflect the technological level at the time, honed and kept sharp by the sweat, tears, and blood given by the previous generations passed down through repetitive training and carefully distilled doctrine given from senior officers to young cadets. Within the military, a commonly used method to make sense and simplify analogies to strategic and tactical decision-making is wargaming. Creating synthetic games that capture aspects of complex warfare in order to practice and prepare future warfare officers for the realities of the maritime battlefield. One of the most enduring analogies throughout the ages is the game of chess.

It is easy to imagine that the original creators of chess have taken inspiration from warfare. Although chess is an abstract turn-based game with a discrete board and well-defined rules, it has a depth that contains strategic and tactical thinking. Anticipating opponent's moves, planning multi-step sequences, and sacrificing material for strategic advantages all resemble the reality of military operations. A lot of commanders throughout history has recommended and endorsed chess as a mental preparation tool for sharpening foresight, disciple, and creativity in warfare.

One of those commanders was Napoleon Bonaparte, a relatively accomplished chess player of his time and arguable the most accomplished military general of all time. He applied the basic principles found in Sun Tzu's *The Art of War* to the battlefield and highlighted the importance of deception and concentration of forces. His contribution to chess theory is the *Napoleon Opening*[2], a bold opening that seeks early board control. At the time, this was considered an aggressive but reasonable opening, but modern chess engine analysis has shown it to have too many weaknesses and have been completely unplayable at the high-levels of chess.

It is but an analogy, few armies actually march in straight lines on neat squares limited to a 8-by-8 resolution battlefield. Modern naval engagements take place in environments of limited information space, using a game board spanning entire oceans, and the amount of possible actions is both enormous, often continuous, and involve multiple domains. Real conflict carries real consequences, making experimentation with new tactics a costly endeavor, with errors requiring payment in lives lost, material and strategic objectives. A chess loss will at best result in a rematch and at worst quitting the game.

However, chess remains a useful metaphoric tool for teaching officers the mental agility necessary to handle the complex dynamics of warfare. Walking in the same footsteps, I will likewise use chess as an analogy throughout this thesis when I bridge the gap from reinforcement learning into naval warfare.

[2] e4, e5, Qf3

It will provide an insightful bridge pier, where the rules are well understood, the environment clear, and the consequences obvious, before diving into the depths of unfamiliar maritime environment.

The Age of the Machine

On the eleventh of may 1997 chess grandmaster Gerry Kasparov, arguably the greatest chess player of all time, faced his last decisive battle against IBM's Deep Blue chess computer. In was in his sixth match with a score of 2 wins, 2 losses, and a draw, that Kasparov as black played the Caro-Kann defense, a robust and engine-favored move today. Kasparov tried to deviate from the main opening line and exploit the programming of Deep Blue but Deep Blue struck back and continued to play assertive and efficiently, ultimately forcing Kasparov to resign after only 19 moves.

This marked the end of human supremacy in chess. The moves played by Deep Blue was still explainable to humans, after all its algorithmic approach resembled the mental algorithm a human player makes before deciding on a move. Perceiving the game in an opening, middlegame and endgame phase, looking for subgoals, evaluating board positions and using search control to limit wasting computing power on ridiculous sequences. Essentially, it was built to think like a human chess player, only now boosted by massive computing power. Kasparov had tried to understand and exploit the algorithms of Deep Blue, much like he would with other high-level chess grandmasters, but had ultimately failed.

This marked the beginning of a new era in chess. Advancements in machine learning, especially the use of neural networks, made it possible to build new, smarter chess engines that combined brute-force search techniques with complex board evaluation functions. The top chess engines of today, like Stockfish, not only crush grandmasters without facing the least of resistance, they produce moves so counterintuitive that humans have a hard time explaining them, and why they work. This has fundamentally changed how top players approach the game. Some openings are no longer viable as their weaknesses have been completely mapped by chess engines, and some previously thought to be dead ways of approaching the game have become alive again.

Chess engines are now sparring partners for human chess players, they come up new novel ideas, revealing patterns humans usually miss, and thus augment the training process. This methodology of using computers to analyze positions, evaluate moves, and suggest tactical sequences will throughout the thesis be refered to as *emergent tactics*. These range from subtle to surprising sequences of actions, found by exploiting analysis with computing power, in order to find new tactical approaches to situations that lie far beyond the human calculation horizon.

In chess, this means consistently finding hidden resources, radical sacrifices, dismissed quiet moves, or unorthodox defenses that would otherwise

never be played or found by humans. As a result, chess players can then integrate these new patterns and emergent tactics into their own gamestyle, expanding their repertoire of moves, and boosting the decision-making of their game play.

The implications of the introduction of computing power to boost decision-making have consequences reaching far from the chess board. This transition from human to machine supremacy was most visible in chess and because of how easily *modelable* chess is, the ease of which a game could be modeled computationally, it happened earlier than other games and systems. In other realms of gaming, AI systems such as AlphaGo have shown superhuman play in Go Silver et al. [2017], and even the real-time strategy game of Starcraft II now has AI competitors competing against and even winning against the best players, Vinyals et al. [2019]. These systems use self-play and multi-agent reinforcement learning to improve their play and discover new tactics, without using hard-coded rules like Deep Blue. Thus, mimicking the trial-and-error approach similar to how humans learn from repeated practice.

Boosted Decision-Making in Naval Warfare

Evidently, AI can beat the world's best chess players and even real-time strategy gamers, by employing superhuman tactics. Humans can achieve superiority over others humans, by observing and augmenting their play style through discovered emergent tactics from machine-play. This progress begs the question;

"If AI can boost decision-making for players in games, could a similar approach give advantages in real-world military contexts?"

The military world of today is often divided into *domains* (land, sea, air, space, cyber, etc.) with their respective *warfares* (for the sea domain, typically anti-surface warfare, anti-air warfare, anti-submarine warfare, mine warfare, etc.), each with their own unique characteristics, advantages, challenges, environment, and weaknesses. The separation varies from one military institution to another and form the underlying framework upon which their respective doctrines is built upon. Based on these characteristics, some types of warfare would be better suited for boosted decision-making. The land domain is highly complex, with a seemingly infinite amount of variables for decision-makers to sort through. Sand can get in the eye of a soldier before a critical shot, a weapon can jam because of poor maintenance and interactions with local populous can be unpredictable.

Naval Warfare is different, the open sea can arguably be approximated as flat and feature less, the timeline of decisions is often on the order of minutes or hours where ship maneuverability comparably is almost instant, units are very small compared to engagement distances and therefore the spatial and

temporal resolutions are within an acceptable level. Both naval warfare and games involve adversarial decision-making under logistical constraints, though the scale of naval warfare is massive compared to any board game.

The cost and scale of naval operations and exercises restrict the willingness of navies to experiment with new tactics and puts weight on conservative and doctrinal tactics. This gives further incentive to the use of realistic simulation, as is already seen in the wargaming process of modern navies. The goal is not only to train tactical officers in generalized warfare but also to prepare them for specific engagements. The Royal Danish Navy already uses specialized board games, in order to train operational planning skills in the Danish cadets CMO [2021]. Using computer models, this process may take a massive leap forward, similar to the change that has occurred in the chess community.

If this adversarial wargaming process could be automated and run by computers, using similar techniques as chess engines and AlphaGo, on a virtual naval battlefield, agents could explore new tactical approaches and innovate in manners beyond normal human capacity. If implemented into combat management system, they could augment and boost decision-making for warfare officers.

However, it should be noted that the gap between simulation and reality is nontrivial. In the real maritime world, the oceans obey the physical laws of fluid dynamics and continuum mechanics, and the quirks of human decision-making is certainly not always optimized. Platforms and weapon systems malfunction, disruption of communication, and jammed or even spoofed sensor inputs occur. Political, legal, and moral constraints on naval decision-making may not be easily modellable in simple computations. Dealing with these disconnections and approximations are critical for the applicability of the discovered emergent tactics. None-the-less, the lessons learned from advanced AI playing chess or Starcraft, that superhuman emergent tactics dominate human tactics, remain an exciting prospect inviting further research.

The Modern Dilemma: Human and Machine Reasoning

A central question in how AI could be applied to warfare is how human oversight should be balanced with machine-driven autonomy. When lives are at stake, ethical and accountability concerns inevitably arise. Even partial automation, where AI suggests courses of action for human warfare officers who hold the ultimate decision-making authority, can create a cognitive overload and lead to confusion if the reasoning behind the emergent tactic is not understood, especially in high-stakes situations. The decision-makers must trust not only the tactical output, but also the process by which by machine calculates the tactic. This is seen across the field of AI and is often cited as the *explainability* of an AI system.

Similarly to the struggle chess grandmasters initially had with understanding the logic behind the moves of advanced chess engines, naval decision-

makers may find it hard to accept or implement innovative tactics proposed by AI systems. What will happen when an algorithm suggests a suicidal course of action that holds hidden long-term strategic value? Will the human operators trust it, without understanding the reasoning? From chess we know that the most effective play can look bizarre by conventional wisdom, but reveal its true value later in the game. The risk taken in naval operations is infinitely higher, there is no reset button for the lost sailors onboard a warship.

This is none-the-less precisely the motivation for deeper insight from further research. An intelligently designed machine-learning approach could identify weaknesses in opponents sensor coverage or exploit timings and illustrating windows of opportunity that would otherwise be impossible to notice with conventional wargaming analysis. The synergy from naval experts on operations and automated optimization could help navies adapt to the new era where threats evolve faster than ever before.

Towards an AI-Driven Future

The transition from human to machine decision-making supremacy in chess, is an early case-study for how the AI-driven systems might change other fields of decision-making. The prospect in warfare is exciting, it may become an deterrent, securing peace when machines predict the outcomes of wars before they are initialized. It does also come with a scary potential as an asymmetrical weapon, showing radical non-state powers the hidden weaknesses in larger state powers.

As technology has transformed naval warfare throughout the ages, from sailing vessel with arrays of cannons to modern electronic marvels filled to the brim with sensors and missiles, so has it transformed the world of strategic gaming. The introduction of machine learning and reinforcement learning has made AI a superior opponent in many domains otherwise thought to be exclusive bastions of human ingenuity. The overlap in adversarial wargaming, resource management and anticipation of opponents actions, suggest that naval strategy could benefit from the high-speed, high volume iterative learning process brought fourth with cutting-edge AI algorithms.

I aim to demonstrate in this thesis, that while no direct link between simulation and naval warfare exists, a partial bridge can be built with useful properties. By harnessing new algorithms in machine learning, in particular reinforcement learning, new emergent tactics can be discovered that augment human expertise in warfare. The transition to combined human and machine-driven decision-making is both an opportunity and a challenge for the future planners of naval operations. Through rigorous testing and wise implementation this transition can happen faster and safer.

Problem Statement

In this thesis, I aim to design and implement an adversarial reinforcement learning algorithm, with the goal of using it to anneal emergent tactics from techniques similar to energy minimization. The setup is applied on a subset of naval warfare serving as a case study for its general implementation into other domains and regions of warfare. By doing this, I aim to determine if these algorithms could assist warfare officers with courses of action in real-world situation and to demonstrate how decision-making can be boosted and optimized by machine-discovered emergent tactics augmenting and supplementing human expertise, in high-risk, high-pressure environments. At last I wish to examine how these algorithms could impact the tactical, operational and strategic military levels of the future. In shorter words;

The aim of this thesis is to design and implement an adversarial reinforcement learning approach to explore tactics in a simulated naval warfare environment and determine if this approach can boost decision-making for the modern warfare officer in warfare scenarios.

Research Questions

- **1 -** Define tactics in a mathematical framework.
- **2 -** Build an adversarial reinforcement learning framework for optimizing tactics through learned policies.
- **3 -** Discover can the framework be integrated into a naval simulator.

Approach and Scope

The approach used in this thesis will be on a subset of naval warfare, specifically anti-surface warfare. This subset has the advantage of a simple easy-to-model environment, a temporal resolution on the order of minutes, and spatial resolution much greater than the size of ships, allowing me to approximate them as points. Simply said, each fleet will consist of number of ships with variable speed, a offensive weapon system layout and they attempt to shoot and destroy the opponent fleet.

The questions to be answered and augmented for commanding officers within this warfare subset is then simply;

Where should I maneuver my ships in order to achieve my objectives?

I begin by applying game theory in the context of reinforcement learning, first to chess and then the naval subset, in order to lay a sound mathematical foundation for what I want to achieve.

Then I start constructing a simulated environment that encapsulates the critical aspects of the naval subset model. Two reinforcement learning agents, representing competing naval fleet commanding officers, will be trained from a given start condition, representing a naval scenario. Over time through many iterations their reward functions should converge and stabilize, and a single defensive/offensive tactic for each fleet should be found. The reinforcement learning algorithm used will be the deep deterministic policy gradient. I will constrain the agents actions space by applying naval physics such as realistic ship movement and weapon engagements.

The programming will be done in Python relying heavily on the Torch and TorchRL packages. ChatGPT 4o and o1 is used as a coding debugger.

Thesis Structure

- **Chapter 4: Literature Review** - I examine how the military and research-world have approached the intersection of AI and warfare, with a focus on reinforcement learning and adversarial training.
- **Chapter 5: Mathematics of Tactics** - I lay the mathematical foundation for tactics and define what boosted decision-making is. Then proceed to build the foundations of a naval environment through game theory.
- **Chapter 5: Synthetic Commanders** - I provide the adversarial reinforcement learning framework for synthetic commanders that act in the naval environment.
- **Chapter 6: Results** - I present results from running the the framework on a series of warfare scenarios.
- **Chapter 7: Discussion** - I discuss the finding of the previous chapter.
- **Chapter 8: Conclusion** - Summarizes key findings.
- **Chapter 9: Challenges** - I discuss and present the challenges faced and what methods have failed to work.
- **Chapter 10: Future Research** - I present recommendations for future researchers into the subject.

Chapter 4

Literature Review

FROM Mahan's emphasis on major fleet engagements, capital ships, and concentration of forces Mahan [1890] to Julian Corbett's insistence on naval blockages, commerce raiding and fleet-in-being Corbett [1911], the focal point of naval strategy has ever moved throughout the ages. However the main academic focus has remained on large-scale strategy, sea control and power projection, whereas the tactical elements, meaning those concerned with real-time decision-making on the naval battlefield, have comparatively received little academic attention.

In recent decades a rise in the field of military wargaming, has partially bridged this gap from tactics to strategy. Wargaming has provided decision makers with a simulated environment to experiment and test new tactical and doctrinal ideas, without risking real units or resources Perla [1990]. The simulated environments take many forms, from expanded board games like the early grandfather of all war games "Kriegsspiel" used by the Prussian army in the 19th century, or modern versions like the board games employed at the Royal Danish Defence Academy CMO [2021]. Recently physics-focused high-fidelity digital simulators like *Command: Modern Air/Naval Operations* try to replicate sensor constraints, communication limits, probability, and limited information space.

One major constraint on these traditional wargaming methods are clear;

Limited Exploration: Human or prescipted wargaming agents cannot easily explore the vast decision-space of modern naval warfare. In lack thereof, reverting to assumptions about decision-making based on existing doctrine. This limits the exploration of the wargame sessions to the neighborhood of doctrine.

This constraint gives motivation for reinforcement learning approaches. Reinforcement learning agents do not get tired, can handle massive amounts of simulation in parallel, and search the decision space without the usual

human doctrinal bias. In this chapter, I will examine relevant research in reinforcement learning and its implementation in military context.

Reinforcement Learning Fundamentals

Reinforcement learning is often cited as the third branch of machine learning, along wtih supervised and unsupervised learning. In reinforcement learning, an agent learns by interacting with an environment. The agent observes the state of the environment, decides on an action that produces some change in the environment, which in turn provide a reward to the agent. A sequence of state-action pairs is called a trajectory or an episode, which will have a cumulative reward. The mapping from states to actions are called the policy of agent, which in its learning process over many iterations begins to maximize the cumulative reward for an episode. Classically, the environment is modeled as a Markov Decision process, characterized by the total set of states, a set of possible actions, a transition function linking states with actions and a reward function.

Central reinforcement learning approaches are;

1. **Value-based methods:** Most famous is Q-learning, where the quality of each state-action pair $Q(S, A)$ is stored as a value in a Q-table Watkins and Dayan [1992]. An agent may then look up in the table for the action with the best quality value given a state input. Many different approaches to how that table is updated exist, like Monte Carlo methods, SARSA[1] and Temporal Difference learning S. Sutton and G. Barto [2018]. The downside of the classic methods are that the Q-table quickly increase in size as the state-space grow and the required matrix multiplication is computational heavy to calculate. Another popular value-based method in recent times is the Deep-Q-Networks (DQN) algorithm, which utilizes a neural network in order to predict the Q value given a state input. Instead of maintaining the Q-table, the policy information is implicitly stored in the neural network's weights Mnih et al. [2013].

2. **Policy-based methods:** Works by updating the policy, which is the mapping between state and action. Instead of learning a Q-table, the agents learn a stochastic policy $\mu(a|s)$ mapping states to actions with probabilities. First demonstrated with the REINFORCE algorithm introduced in 1992, which learns by improving the policy based on the log-likelihood of gradient of the expected return Williams [1992]. Later, advanced methods appeared, such as Proximal Policy Approximation, which improve stability by imposing constraints on policy updates Schulman et al. [2017]. The advantage of policy-based methods over value-based methods particularly lie in the ability to model continuous action

[1] Abbreviation for State-action-reward-state-action.

and state spaces, whereas Q-learning and DQN struggle as they require discrete actions for their neural network output.

3. **Actor-Critic methods:** Represent the best of both worlds by combing elements of both value-based and policy-based approaches. An actor which learns the policy $\mu(a|s)$ and a critic which evaluates the value function to guide policy updates $Q(S, A)$. This structure reduces variance when updating the policy while maintaining the ability to learn stochastic policies. First proposed by Konda and Tsitsiklis Konda and Tsitsiklis [1999] and later improved with neural networks with algorithms as Advantage Actor-Critic (A2C), Asynchronous Advantage Actor-Critic (A3C) Mnih et al. [2016] and Deep Deterministic Policy Gradient (DDPG) Lillicrap et al. [2015]. Recent advances include the Soft Actor-Critic (SAC), which uses entropy maximization to improve exploration Haarnoja et al. [2018].

These methods should not be confused with on-policy and off-policy methods. On-policy methods such as SARSA, REINFORCE or PPO update their value function and policies based on the reward for actions taken by the current policy. Whereas Off-policy methods like DQN, DDPG and SAC update their policy using a variety of different methods - most notably by using a replay buffer containing past trajectories.

Most suitable for investigation in this thesis is Actor-Critic methods, since they are suitable for continuous state- and action spaces.

Adversarial Approaches in Reinforcement Learning

A natural expansion of single agent reinforcement learning is the addition of multiple agents in Multi-Agent Reinforcement Learning (MARL). The settings for MARL can be competitive or cooperative, collaborating with each other through a shared or mixed reward function, or competing against each other with separate reward functions. Algorithms such as Multi-Agent Deep Deterministic Policy Gradient (MADDPG) Lowe et al. [2017] or QMIX Rashid et al. [2018] accomplishes cooperation with joint decision-making by mixed reward functions.

Cooperative learning is an interesting case for multi-state warfare where more than 2 decision-makers are present or where the goals are asymmetric, and the interaction between agents is not a zero-sum game. However, this is beyond the scope of this thesis, and I will concentrate on true competitive adversarial reinforcement learning algorithms.

In the adversarial world, competitive learning occurs where agents optimize strategies that incorporate the adversary strategy. This strategic interplay creates a feedback loop which is the source of the novel emergent tactics, recently displayed with Google's AlphaGo systems in both chess, shogi the ancient chinese game of Go Silver et al. [2017]. Common for all of these games

are that their game-tree is discrete and well-defined, allowing for easy tree search algorithms. In naval warfare that luxury do not exist and inspiration must be taken elsewhere, for example the team that achieved the grandmaster level in StarCraft II, a real-time strategy game Vinyals et al. [2019].

By using multi-agent reinforcement learning and initially training the agents on recorded human play, they were able to achieve grandmaster level competitively in human leagues. This was achieved by "feeding" human game transitions into the training replay buffer. This highlights the use of a premade replay buffer with human play to initialize the training when the decision-space is incredibly vast. A similar approach will be tested in this thesis.

Applications to Military Context

The first description of the use of computers to support decision-making in the military can be dated back to operations research during World War II. Here, computers provided analytical methods for firing ballistics, resource logistics, and even certain aspects of tactical engagements, as summarized by Philip Morse in *Methods of Operations Research* Morse [1951].

Over time, the role of computers became more and more important and soon provided assistance in many different areas, from submarine underwater detection, automated radar plotting, missile early warning detection, and even simulating environments for training pilots, Arar and Ayan [2013]. With the introduction of high-performance computing, machine learning libraries, accessibility of chips optimized for machine learning, research in defense-oriented applications for reinforcement learning has risen since the beginning of the 2020's.

The RAND corporation[2] issued two reports on the utility and risk associated with reinforcement learning in military context, Briggs [2024], Bunch et al. [2024]. Most notably, they found that the US Airforce AFSIM framework[3] was capable of automated wargaming with reinforcement learning and managed attempts at penetrating an opponents static air defence, though they note that;

"The Department of Defense is likely limited in the use and development of RL because of the lack of specialized skill sets in this field and the difficulty in retaining personnel with such skills once obtained, given the highly competitive and lucrative nature of the field."

In addition, the AFSIM framework is highly classified and unavailable to the public, and its use case is in the cyber, air, and space warfare domains. Publicly available information hints at that it does not utilize true adversarial approaches and relies on static or prescripted opponents. This provides fertile

[2]Self-described as: *"... a nonprofit, nonpartisan research organization that provides leaders with the information they need to make evidence-based decisions."*

[3]Advanced Framework for Simulation, Integration and Modeling

grounds for further investigation of similar systems for researchers outside of the US and its expansion into other warfares, in my case the naval domain.

Other military institutions or corporations may already be conducting research in similar areas, but due to its sensitive nature, its highly likely that it would be classified and unknown to the public.

Chapter 5

The Mathematics of Tactics

Symbol	Description	Type
	General Game-Theoretic Notation	
N	Set of players, e.g., {White, Black}	Set
$\mathbf{S}$	Full state space	Set
S_{term}	Set of terminal states	Subset
s, s_t, s'	Current, indexed, and next state	State
s_0	Initial game state	State
$\mathbf{S}_0$	Set of initial game states	Set
a, a_t	Action at time t	Action
$A(s)$	Set of legal actions from s	Set
$F_{\text{trans}}(s, a)$	Transition function	Function
$P(s_{i,n})$	Player function (whose turn)	Function
$u_i(s)$	Utility function (game-theoretic payoff)	Function
$V(s)$	Estimated evaluation function	Function
t	Index over time steps	Integer
	Naval Simulator Notation	
(x_i, y_i)	Position of ship i in 2D space	Vector
$\vec{v}_i = (v_x, v_y)$	Velocity of ship i	Vector
h_i	Heading angle of ship i	Scalar
$\vec{\sigma}_i$	Status vector of ship i	Vector
$\vec{\sigma}_{\text{fleet}}$	Concatenated fleet status vector	Vector
$\mathbf{A}$	Action space	Set
$\mathbf{S}$	State space	Set
$M_{\max}$	Max missiles fired per salvo	Constant
$F_{\text{threshold}}$	Firing threshold	Constant

Table 5.1: Summary of symbols, parameters, and variables used throughout the chapter.

In this chaper, I develop a theoretical foundation for tactics and define boosted decision-making. I begin by analyzing tactics in context of the game of chess using game theory, and then I explore how those concepts can be applicable the naval equivalent of a chess engine. Below is a common reference table for all future chapters regarding mathematical notation.

The Game Tree of Chess

Imagine two players engaging in a game of chess, the set of players would be;

$$N = \{\text{White, Black}\}$$

All possible board configurations (positions) that can arise throughout the game constitute a set of states called the state-space:

$$S = \big\{\, s \mid s \text{ is a legal chess position} \big\}.$$

The game begins in a standard configuration of pieces:

$$s_0 \in S, \quad \text{where } s_0 \text{ is the starting state.}$$

From any state $s \in S$, each player has hoice to make. They must pick an action from a set of legal moves:

$$A(s) \;=\; \{\text{all legal moves from state } s\}.$$

When a player takes an action $a \in A(s)$, the state of the game is altered to a new state s'. This transition is denoted by a transition function F_{trans}:

$$F_{trans}(s, a) \;=\; s',$$

where s' is determined by the rules of chess, for example the limitations of piece movement.

Chess proceeds in alternating turns, starting with White. By indexing each turn by an integer $n \in \{0, 1, 2, \dots\}$, where $n = 0$ corresponds to White's first move, $n = 1$ to Black's first move, and so on. Let $s_{i,n}$ be the state of the game at turn n and described by board configuration i. Then the *player function* P assigning moveorder, can be defined by

$$P(s_{i,n}) = \begin{cases} \text{White}, & \text{if } n \text{ is even,} \\ \text{Black}, & \text{if } n \text{ is odd.} \end{cases}$$

The game proceeds until an *end condition* is reached. In chess there are 3 ways to end the game:

- **Checkmate**: The king is being checked by another piece and cannot escape in the next move.

- **Draw Situations**:
  - *Stalemate*: A player has no more legal moves and their king is not in check.
  - *Agreement*: Both players agree to a draw.
  - *Repetition*: Players repeatedly move back and forward.
- **Resignation**: A player voluntarily admits defeats.

Any complete sequence of moves by White and Black, starting from the initial state s_0 and ending in some terminal state $s_{\text{term}} \in S_{\text{term}}$, forms a *trajectory* or *episode* of the game. In this thesis I define a given trajectory as t and write this sequence as:

$$t = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \cdots \xrightarrow{a_{k-1}} s_k,$$

where each action a_j is chosen from the set of legal moves $A(s_j)$, and $s_k \in S_{\text{term}}$ is a terminal position.

Using graphs, this game tree can be visualized where each state of the game is a node and an action is an edge leading to another node, as seen in figure 5.1. The game rules determine which nodes are linked, by their respective legal moves. The ultimate end for each player is to find a node representing a terminal state, where a satisfactory end condition is met. Each complete sequences of moves from both players resulting in a series of transitions from the starting node to a node with the winning end condition, constitutes a game trajectory.

For chess the scale of this graph is incredibly big. The total amount of possible legal states in chess was estimated by Claude Shannon, Shannon [1950], to be around

$$|\mathbf{S}| \approx 3.7 * 10^{43}$$

Combining that with an average branching factor, or the average amount of legal moves for a given state, of 35 and an average game length of 80 gives *Shannon's Number*, an estimate for the lower bound for all the possible game trajectories that follow the initial starting condition within the allowed rules. [1]

$$\mathbf{T} \leq 10^{120}, \quad \text{where all legal } t \in \mathbf{T}$$

This is a famous approximate measure of the game tree complexity of chess.

Just to get a grasp of the required computational power to compute this, consider the Landauer limit Landauer [1961] for a perfect computer in interstellar space. The Landauer principle describes the minimum amount of

[1] It is an old approximation and more accurate estimations exists today, I've included this one for its ease of understanding.

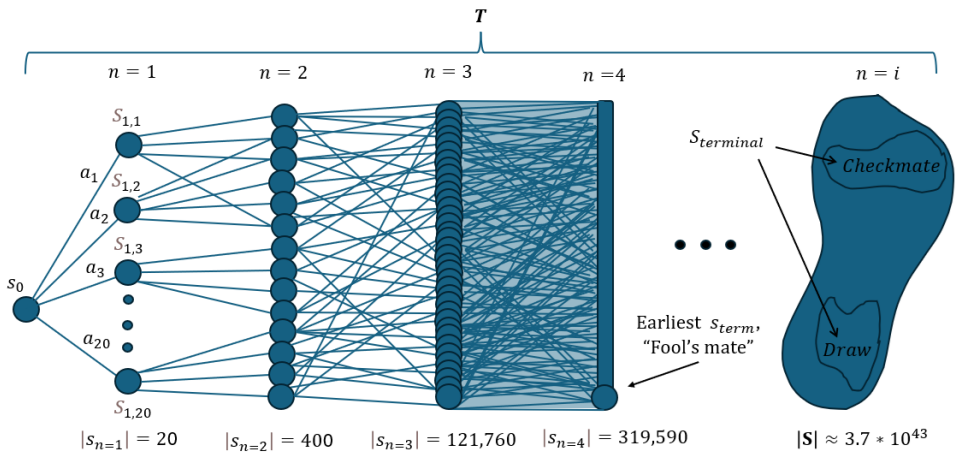


Figure 5.1: Visualization of the game tree of chess. All games start from the S_0 state, and the tree rapidly branches out and becomes big. n denotes the turn, the dots denote states and the edges represents actions. For $n = i$ the state space is drawn as an abstract plane, where every point represents a state. The game ends in a checkmate or a draw at a $S_{terminal}$ state.

energy through heat dispersion needed to erase one bit of information[2] as;

$$E_{min} = k_B T \ln(2) = k_B \cdot 2.725K \cdot \ln(2) \approx 210^{-23}\text{J}$$

Imagine that if computing a chess trajectory only required managing one bit of information and compare the orders of magnitude that to the estimate of the total amount of energy in the observable universe

$$\frac{E_{uni}}{\mathbf{T}E_{min}} = \frac{10^{69}}{10^{120} \times 10^{-23}} = 10^{-30}$$

That is 30 orders of magnitude too short to be computed by any known means, meaning chess is literally incomputable within our understanding of the universe.

Chess Tactics

When an end condition is met, the game is terminated. A subset of *terminal states* $S_{\text{term}} \subseteq S$ is defined to contain all positions $s \in S$ that satisfy one of these end conditions. In a game-theoretic context, its described the result via

[2]only assumed heated by the cosmic microwave background radiation.

a utility function:

$$u_i(s) \;=\; \begin{cases} 1, & \text{if player } i \text{ wins at } s, \\ 0, & \text{if the game is a draw at } s, \\ -1, & \text{if player } i \text{ loses at } s. \end{cases}$$

Here i can be either *White* or *Black*. Because chess is traditionally viewed as a *zero-sum* game[3], the outcome for one player is the negative of the outcome for the other:

$$u_{\text{White}}(s) \;=\; -\,u_{\text{Black}}(s).$$

Since Zermelo's theorem states that all chess game states must exist in either a draw or forced win state Zermelo [1913], one simply needs to calculate all trajectories from a given state and from there infer the utility as 1, 0 or -1. However, Shannon's number is so large that the underlying game tree complexity becomes computationally impossible to search every possibility for each turn in a typical chess position. This is of cause also the root of the desirability of chess; The scale of the game tree is overwhelming and from that arises unpredictability, all while maintaining an easy to understand environment and set of rules.

This underlines the need of human players to direct their algorithmic search for optimal positions. Trajectories are not equal, trading your queen for a pawn would not make sense, unless the pawn otherwise would be unavoidably up for promotion. Humans beat humans by their knowledge and experience with the game, having seen the same patterns of states and actions play-out countless times before.

Instead, both human players and chess engines alike rely on *heuristic evaluation functions* for nonterminal intermediate positions. These functions estimate how favorable a position is for a given player, based on features such as:

- *Material Balance*: Counting the piece of both players and assigning them a value.
- *Piece Activity*: Availability and flexibility of piece moves, how much control does each player assert on the board.
- *King Safety*: Exposure of the king to checks and threats from the opponent.
- *Pawn Structure*: Weaknesses such as isolated pawns, passed pawns, or doubled pawns.

[3] It should be noted that the design of some competitions, where advancement in leagues happens based on the result of multiple games, effectively can change this.

- *Positional Factors*:Such as control of the center, setting knights on outposts or sniping with a bishop pair.

For human players, this usually means counting material points for White and Black, and weighing the difference against the advantage of a position. They train their analytical skills to be sharp and fast, looking for "Checks, Captures, Threats"[4] in order to evaluate their current state and possible transitional states. They improve their memory and are able to calculate many moves ahead, using their experience to draw conclusions about entire branches of the game tree before making their decisions. All of these help them to navigate the gigantic game tree, where they try to force the opponent into unknown state branches where the opponent has less experience or preparation, and thus their search algorithm for optimal moves and positions less efficient.

These recurring patterns are often referred to as a tactic. Specifically speaking, within the context of chess *a tactic would be a set of moves made to a specific position or set of positions, with a defined set of end-positions with a higher estimated evaluation value in mind.* They can thus constitute a single game trajectory (the Scholar's Mate for example), or be a subset of a trajectory (such as a king/queen knight fork or, more subtlety, with the Fianchetto bishops controlling the powerful diagonals).

The complexity of adversarial tactics then becomes apparent when it is considered that one player tactics may be exploited by their opponent, who recognizes the weaknesses and has broader tactical awareness on the chessboard. This arises because tactics are either taught or self-learned through explicit examples and then taken to be generalized. However, the complexity of chess means that a single piece out of place or a hidden set of superior trajectories can cripple a tactic and allow a more knowledgeable player to take advantage of the situation. Tactics are often communicated in simple sentences or by single trajectory examples that fail to capture all the complexity because of the list of exceptions and inclusions necessary for a full description is too large. Shannons number is simply so enormous that often no complete description of a general tactic is possible, because the subset of positions that a tactic may apply on is simply too great for any meaningful interpretation.

The description of tactics is therefore an integral part of how humans employ tactics, and human tactics are by their nature social constructs representing a description of an abstract set of subtrajectories. A tactic with a more complete description through superior game knowledge or greater mental computational ability can defeat another player using a tactic with less. An visualization of this can be seen in figure 5.2.

[4] An example for beginners widely used by chess coaches around the world.

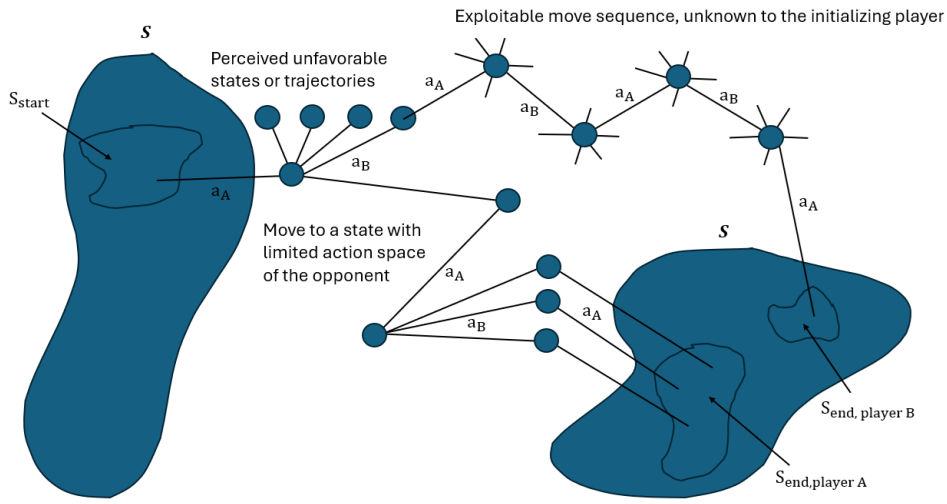


Figure 5.2: A graph visualization of the mental gymnastics a professional chess player might go through before deciding on a move. $a_{A,B}$ denotes which player makes the move. Both players are employing this sequence exploitation and try to outthink the opponent.

More generally in the context of game theory;

Definition: A *Tactic* is a set of trajectories, found by executing actions while anticipating the opponents actions, which can be applied to a set of starting states S_{start} with the intent of arriving at a set of end states S_{end} with a higher estimated evaluation;

$$\{F_{nested}(S_{start}, a_{sequence})\} =$$

$$\{F(F...(F(s, a_1)..., a_2), a_i)| \text{ for all } s \in S_{start}\} \rightarrow S_{end}$$

$$\text{where } V(S_{start}) < V(S_{end}), \quad \{S_{start}, S_{end}\} \subseteq \mathbf{S}$$

Tactics are the mathematical continuation of the action object in an adversarial context. Through this framework they give an understanding as to how a player navigates the state-space by applying a sequence of actions that move from an initial state $s_{start} \in S_{start}$ toward a more favorable state $s_{end} \in S_{end}$, while anticipating the opponents actions. A *favorable* state is defined here by having a higher evaluation value, indicating that the end state is advantageous compared to the starting state. In practice, V could be as simple as a material count in chess.

A general tactic can be applicable to many similar situations, which is why the definition is a set actions applied to a set of starting and end states. An

example could be the Fiancetto bishops in chess, a tactic with a lot of nuance, as it only involves a small section of the board and its use depends on the configuration of the rest of the pieces. The applicable starting states will not include all the states where the move is possible, as the resulting end states might not be of real higher evaluation value.

It is interesting that a tactic can lead to many different outcomes depending on the opponents' counterplay. The mapping from one state to another is not always invertible. In adversarial scenarios, the actions of the opponent is an integral part of the trajectory which increases the complexity of predicting outcomes.

In game theory, tactics can be explored via search algorithms (like minimax with alpha-beta pruning). All the trajectories can be visualized as a as a game tree, where each node is a state, each edge is an action. The evaluation function V estimates the value of each node's and identifying and applying a Tactic amounts to choosing a subtree that leads from s_{start} to s_{end} with the highest payoff under the assumption of optimal or near-optimal play from the opponent or through exploitative back-up plans should the opponent play non-optimal.

However, computational methods are constrained by the *state explosion problem*: even moderately complex games can produce exponentially large state spaces when their size is increased, making complete searches impractical or downright impossible, Valmari [1998]. Humans rely on rules of thumb and domain-specific knowledge to direct their scanning of the state space in order to identify a tactic quickly.

I'll refrain from separating tactics and strategies, and simply let tactics be both long and short trajectories. Not to be confused with the traditional military concepts of tactical, operational, and strategic *levels* of warfare analysis, which will be used in later chapters.

Tactics are the foundation of adversarial competition. They represent the intent of the players to force transitions towards a desired end goal, guided by an evaluation function, and are influenced by potential counterplay by the opponent. Despite their mathematical framework, tactics rely on imperfect human or machine estimates of value and incomplete knowledge of future states. The dynamics of adversarial opponents ensures that tactics remain adaptable tools rather than perfectly deterministic mappings from S_{start} to S_{end}. Understanding when and how to adapt an employed tactic in light of new information or a sudden environmental change is an important skill for gamer and warfare officers alike.

An example from the real world is the battle of Trafalgar. The British fleet led by Admiral Horatio Nelson attempted to blockage the Franco-Spanish fleet from exiting the Mediterranean and gain control of the English channel. The British were massively outnumbered in both ships and manpower, 17000 sailors against 33000. On the morning of October 21, 1805, off Cape Trafalgar, the fleets would engage. The Franco-Spanish fleet sailed in formation 1,

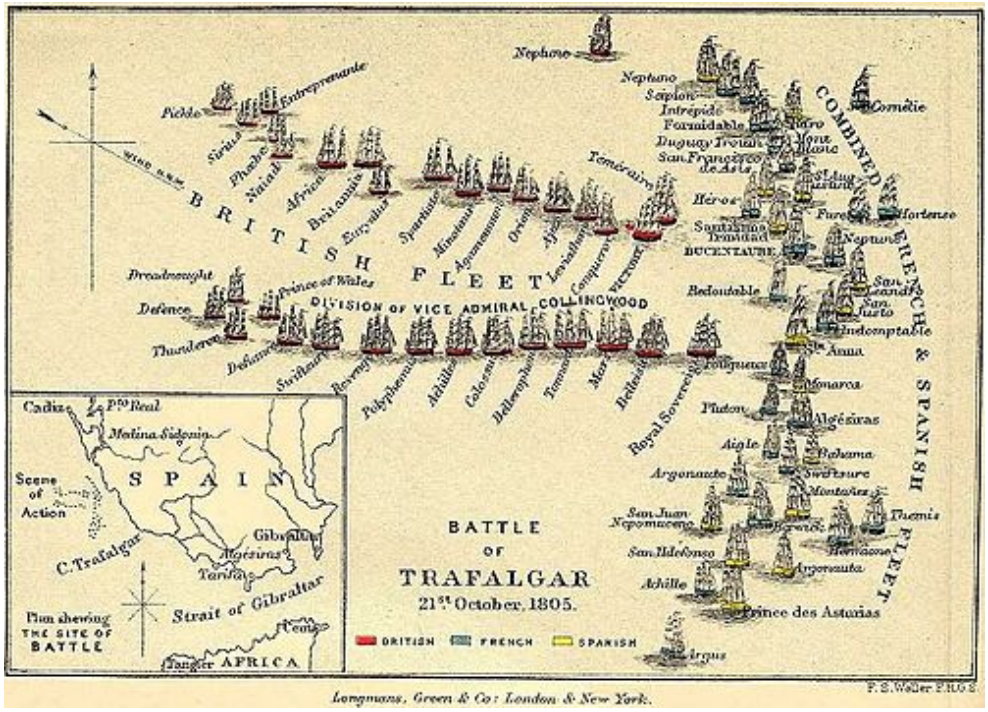


Figure 5.3: The famous battle of Trafalgar, where Admiral Nelson defeated the Franco-Spanish fleet and prevented Napoleon from taking the British isles.

and followed normal doctrine attempting to circumvent the British fleet stuck further out at sea, and thus crossing their T. However, Nelson recognized that their formation was loose and uncoordinated.

Through his superior knowledge of the battlefield and his opponent, he ordered his fleet into two columns and set their course directly into the Franco-Spanish cannon range. In this fashion he was able to effectively double-cross their T and close the distance to the enemy before they acted. It was a bold and risky tactic, but it managed to exploit weaknesses in the opponent tactic through a narrow recognized window of opportunity. Had the Franco-Spanish fleet succeeded, they would have been able to gain control of the English channel, and Napoleon could escort his invasion force to British shores. Nelson lost his life, but his superior tactic, breaking from normal doctrine, had decimated the Franco-Spanish fleet, saving the British islands from Napoleonic invasion. An visualization can be seen in figure 5.3

A Game of Warships

Lets consider the mathematics of a general naval war simulator in the context of game theory and then try to apply the definition of tactics to that.

Core-assumptions;

1. The naval environment can be simplified to a 2D continuous plane.[5]

2. Warships are be modeled as points that interact with each other and the environment.[6]

3. Interactions are stochastic in nature, and the success of a missile engagement, will be modeled by random chance corresponding to a hit-rate inspired from real-engagements.

In this simulation each player commands a fleet of warships. The state of the game is defined by their position denoted by a coordinate pair (x, y) in a continuous 2 dimensional plane.

$$(x, y) \in \mathbb{R}^2.$$

Additionally, each warship has a status vector $\vec{\sigma}$ that describes discrete internal flags such as "Firing threshold", "Targeting", or "Dead/Alive".

The state s at time t is then defined as

$$s_t = \{((x_i, y_i), \sigma_i) \mid i = 1, 2, \ldots, N\},$$

The state is thus composed of both discrete and continuous values.

Each player act simultaneously in the game and the player function outputs them both each step.

The action space is defined as

- **Continuous**: Adjusting the heading h of a ship, chosing a speed v
- **Discrete**: How many missiles to fire and who to target.

The character of the action space then is;

$$\mathbf{A} = \mathbf{A}_{\text{continuous}} \times \mathbf{A}_{\text{discrete}},$$

Of importance is the fact that dual spaces of discrete and continuous values are used and as the number of ships N increases, the state space increases in size rapidly.

[5] 3 dimensions is required if the air and subsurface domains are to be included, for simplicity I just use 2 dimensions. Weather impact on speed, radar coverage, detection probabilities, etc. influence tactics and decision-making, but is ultimately beyond the scope of this thesis. I will return to it later and discuss its implementation in more advanced models.

[6] As the scale of the environment which must encompass missile engagements it is much greater than the size of ships, justifying this assumption. (missile range $\approx$ 100km $\gg$ 100m $\approx$ ship length)

The transition function links states and action together, forming the naval physics of the simulation. Once a player chooses an action $\alpha_t \in \mathbf{A}$, the transition function

$$s_{t+1} \;=\; F\big(s_t, \alpha_t\big)$$

computes the new state s_{t+1}. A simplified sketch of the transition include:

1. **Integrate** continuous actions like movement of ships.
2. **Apply** discrete actions like firing mechanics.
3. **Resolve** application of damage, adjusting dead/alive statuses.

As in chess, each player aims for states in the simulation that they estimate as advantageous. But the description of value $V(S)$ is now more complex. The evaluation $V(S)$ of a given state S could be a weighted sum or other combination of discrete and continuous factors like:

$$V(S) = w_1 \cdot \Big(\text{number of ships left}\Big) \;+\; w_2 \cdot \Big(\text{positional advantages}\Big) \;+\; \dots$$

with weights $\{w_1, w_2, \dots\}$ reflecting the player's tactical priorities.

Unlike chess, a naval simulator need not be a zero-sum game. For ease of implementation, a "deathmatch" style simulator is used throughout this thesis, but the strategic goals on the battlefield do not need to be outright destruction of the opponent. In fact, the desired outcomes for engaging fleets may not necessarily only be adversarial, it can even have cooperative components. A strategic effect often discussed and historically desired is Sea Control or Sea Denial[7].

Imagine that one fleet wishes to achieve Sea Control, and another fleet wishes to achieve Sea Denial in a region. Modeling this means having asymmetric evaluation functions for each fleet that rewards behaviors that align with their strategic goals. The fleet desiring Sea Control could be rewarded for maintaining physical visible presence in the area, escorting high-value units, and denying the enemy access. The opponent fleet desiring Sea Denial, does not care for their access unless it helps them achieve their goal of disrupting enemy logistics or executing asymmetric attacks. If the opponent fleet does not have interest in the area beyond escort, the evaluation function should reward less risky behavior and, if possible, even help guide the fleet around the area of sea control, in order to avoid an engagement. The underlines

[7] *Sea Control* is the ability to exert dominance over a bounded maritime area through complete naval supremacy allowing freedom of movement for friendly and protection of supply lines, while denying it to the enemy. This is done with traditional naval units like destroyers, frigates or corvettes. *Sea Denial* is the ability to deny your enemy access to a bounded maritime area, without maintaining full control over accessability for your own forces. This is done with asymmetric naval threats such as submarines, mines, or land-based anti-ship missiles.

the importance of precise modeling of the fleets strategic goals in the evaluation function and terminating end states, in order to accommodate diverse non-zero sum simulations.

In chess, the players engage in a game with perfect information. All piece positions on the board are known to both players, and the game is fully deterministic. The real world is much less predictable. enemy forces may hide behind the horizon, their emission policy may hinder them practically invisible until very close for missile engagement, and a missile strike involves an uncountable amount of variables like crew fatigue, air defence system setup and maintenance, the seeker type and the local environment of the target in order to predict the outcome. This can be added to the simulation by limiting the information space available to each fleet in the environment. This allows for deceptive and novel tactics, while giving the chance to conduct statistical analysis on an ensemble of trajectories, more on this later.

I extend the concept of a *tactic* to the game of warships by recognizing that a "set of trajectories" now involves maneuvers in continuous space combined with discrete engagements.

In practice, these might involve discrete decisions such as targeting an enemy ship combined with a weapon system while exploiting angles of attack or distances in 2D space. A successful warship tactic in this model might therefore incorporate:

1. **Positioning** in continuous space to optimize firing arcs and screen[8] critical units

2. **Timing** of discrete engagements, such as an pincer maneuver or exploitation of a short window of opportunity.

3. **Resource management** of fleet stockpile of missiles or ship count to sustain longer or multiple engagements.

In chess, a single miscalculation can sink a plan, and likewise small deviations in ship maneuvers can obstruct an entire operation. This is amplified by the combination of continuous and discrete spaces.

Human players in the real world rely on simpler guidelines ("avoid radar arcs," "approach in a pincer formation") to reduce the search space. These guidelines can however be exploited if the opponent anticipates them. Likewise, describing a warship tactic fully can become difficult because accounting for every possible variation in continuous coordinates, speeds, or headings is a massive task. Communication and teaching of tactics requires balance between describing the involved basic principles and acknowledging the limitations of simplified explanation.

[8] *Screening* is the act of positioning a defensive unit between a high valuable unit and the perceived threat

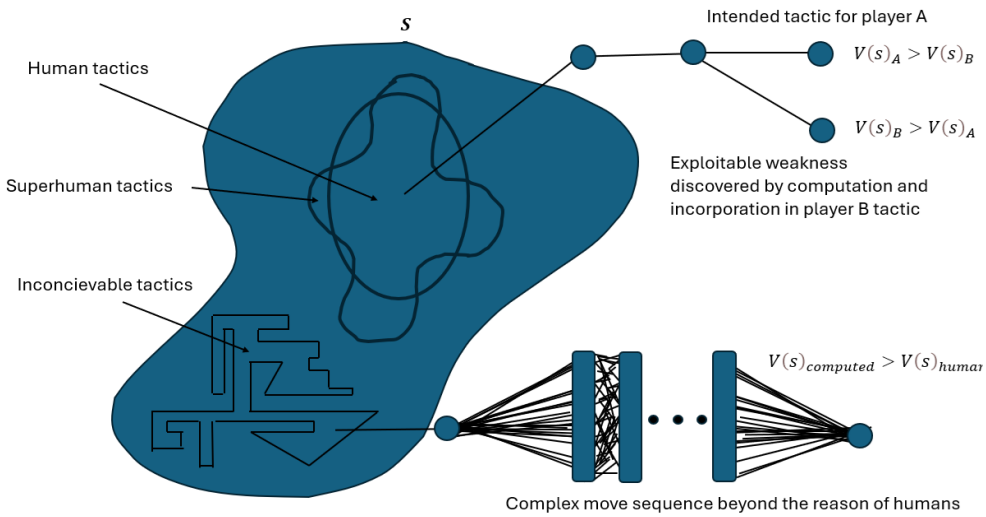


Figure 5.4: Visualization of tactical categories. For the game of warship it should be noted that actions and state changes happens both continuously and discretely.

Tactical Categories

It is convenient to categorize tactics into 3 main categories as seen in figure 5.4;

1. **Human Tactics:** What we normally interpret as tactics. They are self-explanatorily, tactics made by humans. Chess examples include the Fiancetto Bishop, the Shoemaker Check, and the Napoleonic Opening are all examples of tactics made by humans. They are found by mental calculation on a single or a set of starting states, and consist of a subtree of preplanned moves while anticipating the opponents, with the goal of arriving at a single or set of states, with higher estimated evaluation value. The reasoning behind human tactics is based on written arguments or exemplified by demonstrating sample trajectories, highlighting the principles in use. The very description of the tactic may cloud the nuances and its applicability - The applicable set of states S_{start} may not be well understood from a single sentence or one example, therein lies the human aspect of these tactics - and their imperfections.

2. **Superhuman Tactics:** With the arrival of chess engines in the chess world, players began to use engines to precisely estimate the evaluation value of a given board position, predict the best sequences of moves and conduct statistical analysis the big databases of recorded human games.

By learning the precise evaluation value and following the predicted best sequences from given states, chess players managed to decipher, reverse engineer and "make-sense" of the logic the chess engines employed. This allowed them to find weakness to exploit in tactics normally thought to be sound and revive long forgotten tactics thought to be obsolete. Finding new novel move sequences that until not was not thought of. Additionally, by analyzing the ensemble of human played games combined with accurate evaluation values, player where able to find common pitfalls and hidden traps to exploit against human players. This practise has continued since the introduction of chess engines, and today a major part of being a great chess player, is learning how to expand ones chess knowledge and proficiency with chess engines. Developing engine interpretation skills is taught alongside opening- , middlegame- and endgame proficiency. The process highlights how machine power has boosted decision-making for modern chess players, and now represent an integral part of how the game is played. Especially impressive considering that chess is an unassisted game, where chess engines has to be left at home and players are left to fight on their own. The same will not be true for the applications of warfare simulators.

3. **Inconceivable Tactics:** Finally we arrive at the inconceivable tactics. In the chess world, these are engine tactics that humans fail to explain, those looking so alien and illogical, that human reasoning fails to grasp the complexity[9]. No single textual description, or range of examples are able to fully explain its applicability, its complicated tree-node navigation and board evaluation. We see this when engines make repeated moves against eachother, like moving a rook left and right on the back rank. The inconceivables are especially seen in engine vs engine matches, their combined individual optimization pushes the scale of which their subtree search has to look to compete with eachother, to the point where it is uninterpretable to humans. We have no explanation, if we had, they would change category to superhuman, yet these tactics would beat any human player in a heart beat. Our lack of understanding, does in no fashion limit their effectiveness on the chess board..

The borders between these categories are not sharply drawn, and tactics can change category as our understanding of them grows. Initially inconceivable tactics can attain interpretation and become superhuman, and the border between human and superhuman depends on how high regard the interpreter has for the former grandmasters who kept their understanding of the game "secret", or was unable to effectively share their game knowledge because of

[9] A demonstration of such tactics can be seen at Top Chess Engine Championship, where engine vs engine competitions can be viewed in realtime.

the limitations of language and the time required to understand all the intricate details. Today, practically all new tactics are formed with the help of engines, and true human tactics are a thing of the past for chess. All serious chess players employ engine assisted training of their mental algorithms.

Optimization in Hybrid Spaces

By using chess engines to evaluate positions, explore subtree branches and discover new tactics chess players augment their normal abilities with a technological boost. If constructed correctly, a naval similator should be able to boost warfare officers in a similar way.

The reason for the early rise of boosted decision-making in chess is related to its well-defined discrete nature with (relatively) limited board complexity. Classical game theoretic approaches like the search algorithm Minimax or Monte Carlo Tree Search worked great, and was simple to implement. A different set of challenges are to be faced in the hybrid space of a naval warfare simulator. The classical methods will fail to work and therefore demand a different computational approach for selecting actions and discovering optimal tactics.

1. **Trajectory Optimization:** In the discrete world of chess, moves occur one by one and form a graph, which players through use of tactics must navigate. On a continuous board in a naval warfare simulator, optimal tactics require computing continuous trajectories that maximize the advantage from friendly firing arcs while hindering the opponents use of theirs.

2. **Opponent Modeling and Counterplay:** Like in chess, the adversarial setting requires that the optimal tactic isn't just a sequence of isolated optimal actions, but one that accounts for the opponents response. If the opponent response is assumed random or suboptimal, that will influence ones own decision-making and may ultimately result in the tactical failure when faced with a competent opponent. The best optimization of tactics therefore necessarily involves anticipating the opponents moves, which in turn must be expected optimized in anticipation to ones own.

3. **Information Uncertainty and Hidden Tactics:** Chess is a game with perfect information, both players can see the entire board and the game is completely deterministic. A naval warfare simulator may include elements of a limited information space, where not all of the environment parameters are visible to all actors, effectively recreating the "Fog of War" so famously described by Clausewitz in his book *On War*, Lowenthal [1977]. On top of that, engagements are in their nature indeterministic and therefore the algorithms must be able to handle

probabilistic modeling. This lack of perfect information is the source of deceptive tactics, which is an inseparable part of modern warfare.

4. **Abstract Asymmetric Objective Optimization:** Chess is a zero-sum game, but employing a similar style of deathmatch in our naval warfare simulaltor, does not capture the full complexity of the strategic objectives actors may have on the battlefield. Modern operational planning includes a variety of strategic effects to be executed by naval units, like Sea Control, Sea Denial, Blockages, Escort etc. On top of that, competing fleets may have asymmetric or non-opposing goals which should be reflecting in separate state value evaluations for each fleet.

5. **Computational feasibility:** Given the infinite number of possible trajectories resulting from a high-dimensional continuous state space, a complete search of all trajectories is impossible. Instead, search incompleteness must be met with evaluation of the optimization method used and critical thinking.

One way to deal with these challenges is through *Adversarial Reinforcement Learning*. In the following chapter I will map out the theory behind Adversarial Reinforcement Learning with a focus on the deep deterministic gradient decent method, discuss how to use it to create synthetic commanders for the game of warships and elaborate on how it deals with these challenges.

Chapter 6

Synthetic Commanders

As mentioned in chapter 4.1, reinforcement learning works by having an *agent* interact with an *environment*. The agent observes the state of the environment, and through a *policy* chooses an action that changes the environment, which in turn produces a reward for the agent. The general reinforcement learning scheme can be seen in figure 6.1. This process continues until a *termination* criteria is met, and constitute a trajectory. The goal for the agent is to find a policy, the complex mapping from state to actions, that maximizes the cumulative reward over the trajectory. It is on this foundation that I will build the synthetic commanders in charge of each fleet in the game of warships. On the next page is table 6.1 with used variables throughout the rest of the chapter.

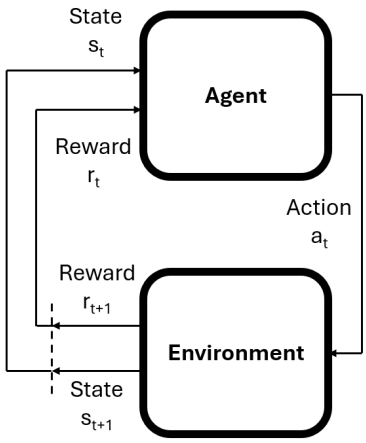


Figure 6.1: General scheme of an reinforcement learning setup. Arrows indicate transfer of actions, states and rewards, dotted line indicates iteration break.

Symbol	Description	Type
Reinforcement Learning Notation		
μ_θ	Actor network (policy)	Function
Q_ϕ	Critic network (action-value)	Function
$\mathcal{D}$	Replay buffer	Set
$\mathcal{B}$	Sampled mini-batch	Set
d	Termination flag	Boolean
γ	Discount factor	Hyperparameter
$\alpha_{\text{actor}}, \alpha_{\text{critic}}$	Learning rates	Hyperparameter
τ	Polyak averaging factor (target update)	Hyperparameter
$\mathcal{N}$	Exploration noise	Distribution
ϵ	Small value to prevent division errors	Constant
Reward Notation		
r, r_i	Reward at time t for ship i	Scalar
$R_{\text{grav}}, R_{\text{attract}}$	Gravity term scaling	Coefficient
$R_{\text{sup}}, w_{\text{sup}}$	Superior range reward parameters	Coefficient
$R_{\text{form}}, D_{\text{form}}$	Formation reward params	Coefficient
$R_{\text{conf}}, \delta_i$	Boundary confinement parameters	Coefficient / Scalar
$R_{\text{pred}}, \vec{d}_j^{\text{future}}$	Predictive intercept reward and predicted position	Coefficient / Vector
$\vec{w}_{\text{fleet}}$	Fleet-wide status reward weights	Vector
$R_{\text{hit}}, R_{\text{status}}, R_{\text{victory}}$	Engagement and status-based reward coefficients	Coefficients

Table 6.1: Summary of symbols, parameters, and variables used throughout the chapter.

The Bellman Equation

A central concept in reinforcement learning is the *value function*, which quantifies the value of a given input state for the agent, in terms of future rewards. The *Bellman Equation* is a recursive formulation that describes the value of a given state in terms of the immediate rewards from the next possible states. For any given policy, denoted μ, and a state s, a state-value function $V^\mu(s)$

describes the expected reward returned from following the policy π[1];

$$V^{\mu}(s) = \mathbf{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \middle| s_0 = s\right] = \mathbf{E}\left[r(s,a) + \gamma V^{\mu}(s)\right]$$

where:

- s is the current state
- $a = \mu(s)$ is the action produced by the policy
- $r(s,a)$ is the returned reward from the environment
- s' is the resultant state from taking action a
- γ is the discount factor, which discounts future rewards in order to favor faster trajectories.

Similarly, the *action-value function* $Q^{\pi}(s,a)$ evaluates the expected return of taking action a in state s:

$$Q^{\mu}(s,a) = \mathbf{E}\left[r(s,a) + \gamma Q^{\mu}(s', \mu(s'))\right]$$

By their recursive structure, these equations allows the agent to learn while a trajectory is being computed without having to wait until a trajectory is terminated, essentially *bootstrapping* its learning to its current trajectory. This will allow the synthetic commanders to learn through trial and error, by continuously computing estimates of how their decisions affect the long-term success of the trajectory.

The Deep Deterministic Policy Gradient Algorithm

A very popular and effective reinforcement learning algorithm is *Deep Deterministic Policy Gradient*. It works great for continuous spaces and is stable for long trajectories, making it suitable for my synthetic commanders[2]. It belongs in a group of algorithms called Actor-Critic's, which exploits both policy-based and value-based methods.

It consists of three main components;

- An **Actor**, which decides on what action to take based on the input state. It approximates a deterministic mapping from a states s to a continuous action vector a based on a policy $\mu_{\theta}(a|s)$, characterized with parameters θ.

[1] By convention π is used to denote the policy when formulating the Bellman equation, but that generally also implies an stochastic policy, whereas μ is used for deterministic policies. Since this is the only time it would come up, I've taken the liberty of sticking with μ.

[2] In challenges I will elaborate on how I've tried to deal with its application to hybrid spaces.

- A **Critic**, evaluates the action taken by the actor by computing the action-value function $Q^{\mu}_{\phi}(s,a)$ parameterized by ϕ and decides how much should be learned from that action.
- Two **Target Networks**, one for the actor denoted μ^{target}_{θ} and one for the critic denoted Q^{target}_{ϕ}. They ensures stable learning by eliminating parametric dependencies with slowly updating parameters compared to the actor and critic neural networks.

For the deep deterministic policy gradient algorithm, these component are neural networks that are continuously updated every step, in order to improve their mappings and ultimately produce better action predictions.

The algorithm starts with the actor observing the environment in state s and choosing action a according to its policy $\mu_{\theta}(s)$. In order to introduce exploration, a noise $\mathcal{N}$ is added to the policy. The action results in the environment changing to state s' and returns the reward r and a termination boolean d.

This transition is then stored as a tuple in a memory buffer $\mathcal{D}$

$$\mathcal{D} = (s, a, r, s', d)$$

The loss function of the critic is set as the mean squared Bellman error, derived from how close the current action-value function $Q(s,a)$ is to the optimized action-value function $Q^*(s,a)$.

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbf{E}_{(s,a,r,s',d)\in\mathcal{D}}\left[\left(Q_{\phi}(s,a) - (r + \gamma(1-d)\max_{a'} Q_{\phi}(s',a')\right)^2\right]$$

The term $r + \gamma(1-d)\max_{a'} Q_{\phi}(s',a')$ denoted y is called the target and the goal of the algorithm is to approximate the current action-value function to this target. Its defined by using the target neural networks for both the actor and the critic.

$$y(r, s', d) = r + \gamma(1-d)Q^{target}_{\phi}(s', \mu^{target}_{\theta}(s'))$$

The critic neural network updates its approximate action-value function by sampling a batch of transitions $\mathcal{B} \subset \mathcal{D}$ and using one-step gradient decent to update parameters ϕ.

$$g_{\phi} = \nabla_{\phi}\mathcal{L}(\phi, \mathcal{B}) = \nabla_{\phi}\frac{1}{|\mathcal{B}|}\sum_{(s,a,r,s',d)\in\mathcal{B}}(Q_{\phi}(s,a) - y(r,s',d))^2, \quad \phi \leftarrow \phi - \alpha_{critic} g_{\phi}$$

and likewise the actor neural network updates its policy parameters θ using one-step gradient decent.

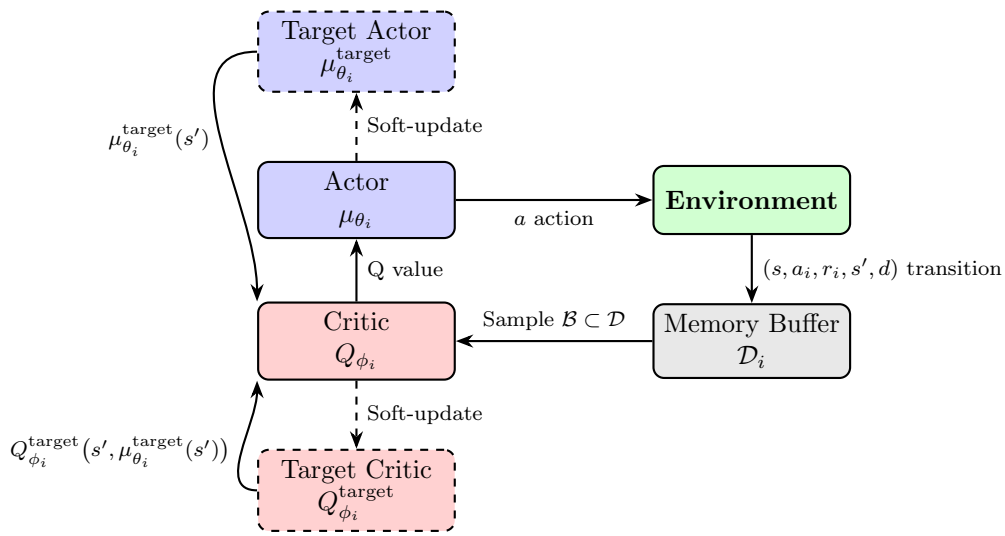
$$g_\theta = \nabla_\phi \mathcal{L}(\phi, \mathcal{B}) = \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} Q_\phi(s, \mu_\theta(s)), \qquad \phi \leftarrow \phi - \alpha_{actor} g_\phi$$

Since the target depends on the same parameters ϕ and θ that are used for the current Q function, this minimization becomes inherently unstable. Therefore, a soft-update is introduced when updating the target neural network parameters ϕ^{target} and θ^{target}.

$$(1-\tau)\phi + \tau\phi^{target} \rightarrow \phi^{target}$$

$$(1-\tau)\theta + \tau\theta^{target} \rightarrow \theta^{target}$$

where the hyperparameter $\tau \in [0, 1]$ represents the size of the delay.



After every step taken by the agent, the neural networks update their parameters

The algorithm continues until a termination criteria is met, at which point the environment is reset and a new episode begins.

Normalization of states and rewards are performed when the transition tuple is stored in the memory buffer. Separate normalization happens for rewards tied to Alice and Bob, as to make them independent.

The memory buffer is not reset with the environment and will contain transitions based on outdated policies. This is in fact a strength of the algorithm, since the transition tuple used to train the neural networks contains the value (reward) of the state-action interaction independent of the policy.

Therefore, it can be trained multiple times on the same transition and gain the benefit/learn the disadvantage many times, reducing the amount of

environment interactions needed for the agent to achieve an optimal policy, a necessary component for realistic convergence in high-dimensional spaces. This effectively makes the algorithm off-policy reinforcement learning.

Lets take an example: Imagine a simple 2-dimensional one agent setup, where the goal of the agent is to reach within a target distance of a fixed goal. The state-action mapping can be visualized by plotting the predicted actions on a grid, forming a flow diagram. The gridpoints represents a subspace of the environmental state-space, which as the agent learns the optimal policy, should approximate taking the direct route as seen in figure 6.2. For more complex problems where the state-space is multidimensional, visualization becomes difficult.

It should here become apparent why it doesn't matter that the memory buffer will be filled with transitions based on old policies, as all transitions merely describes the relation between that state-action pair and the resultant reward. The evolution of the policy and the transitions stored along the way, simply help guide the neural networks to get a focus its approximation of the mapping in the regions where that are most critical for the cumulative reward.

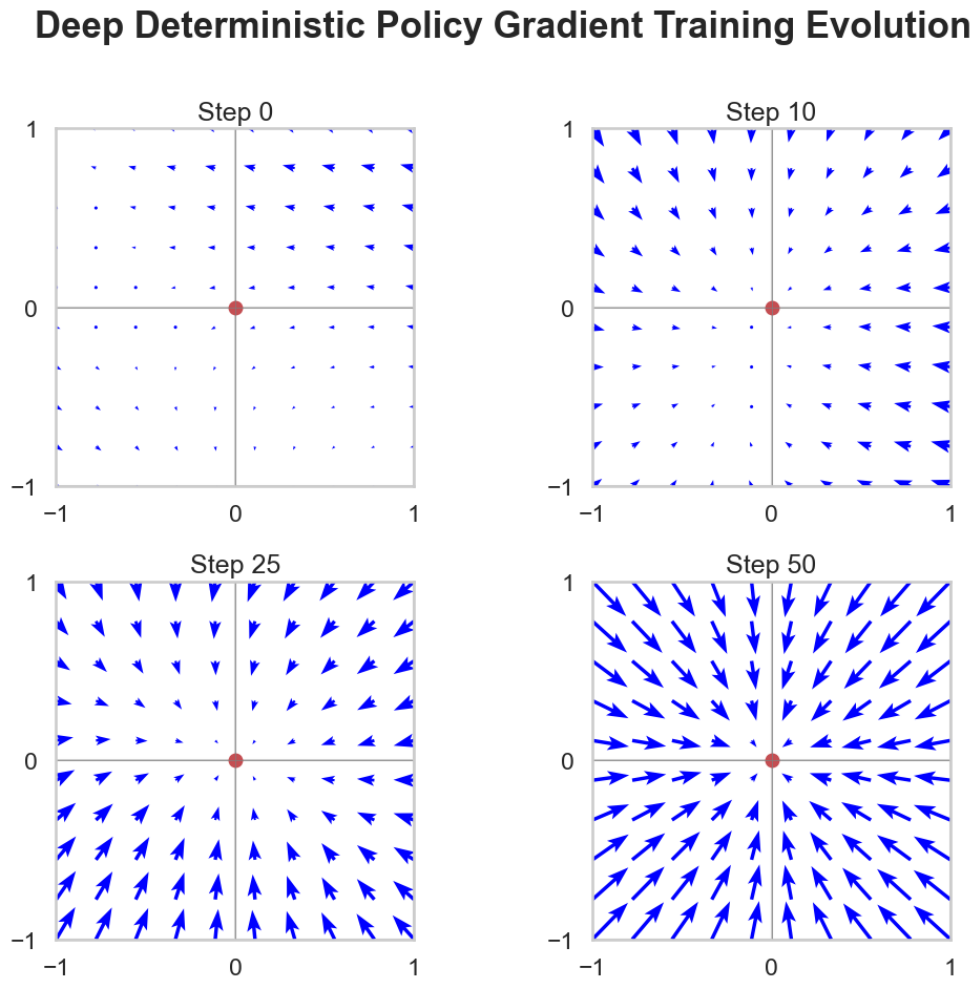


Figure 6.2: The training evolution for a simple 2-dimensional one agent setup, where the agent learns the optimal path to a goal (red dot) in the center. Arrows represent the predicted action from the deep deterministic policy gradient through its training evolution where optimization towards the direct route with maximum speed is chosen. Reward is set as the negative distance to the goal $r = -D$.

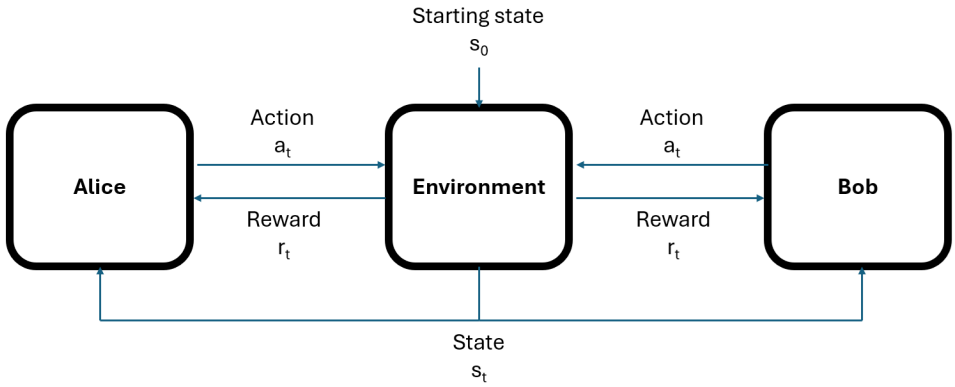


Figure 6.3: Adversarial agent scheme for the multi-agent reinforcement learning setup. Both agents interact with the environment and their policies now becomes dependent on eachother.

Adversarial Commanders

The next step is to introduce the adversarial component of warfare into the simulation. I do this by adding a second pair of actor-critic networks and supporting target networks in the environment, with each set representing the commanding officer of the fleet as seen in figure 6.3. Following in the footsteps of many great physics authors, I name them Alice and Bob. By having two competing agents in the role of commanding officers act on the environment of the naval simulation, I add a new layer of complexity. Now, the agents not only have to optimize against a passive or predictive environment updating its trajectory in isolation, but must do so against another reactive agent. The problem is then shifted into the category of multi-agent reinforcement learning, where there are dependencies between the policies of the interacting agents.

The actor networks of both agents, act at the same time each iteration on the environment, before computing the rewards returned to the critic networks of both agents. In game theoretic context this means that the player function does not shift between players like seen in chess, but allows both players to make moves simultaneously.

In pseudo-code the algorithm can be written as;

Algorithm 1 DDPG for Adversarial Commanders

for Agents $i \in \{1, 2\}$ **do**
  Initialize actor and critic networks μ_{θ_i} and Q_{ϕ_i}
  Initialize target networks: $\mu_{\theta_i}^{target}$ and $Q_{\phi_i}^{target}$
  Initialize memory buffers $\mathcal{D}_i$
end for
for each episode **do**
  Reset environment and observe initial state s
  while $d = 0$ **do**
    Each agent selects action with noise:

$$a_i = \mu_{\theta_i}(s) + \mathcal{N}_i \quad \text{for } i \in \{1, 2\}$$

    Execute joint action (a_1, a_2) in environment changing it from s to s'
    Observe next state s', rewards r_1, r_2, and termination flag d
    Store transition tuple (s, a_i, r_i, s', d) in memory buffer $\mathcal{D}_i$ for each agent i
    for each agent $i \in \{1, 2\}$ **do**
      Sample mini-batch $\mathcal{B}_i \subset \mathcal{D}_i$
      Compute target using target networks:

$$y(r_i, s', d) = r_i + \gamma(1 - d)Q_{\phi_i}^{target}(s', \mu_{\theta_i}^{target}(s'))$$

      Update critic by gradient decent:

$$g_{\phi_i} = \nabla_{\phi_i}\mathcal{L}(\phi_\rangle, \mathcal{B}) = \nabla_{\phi_i}\frac{1}{|\mathcal{B}|}\sum_{(s,a,r,s',d)\in\mathcal{B}}(Q_{\phi_i}(s, a) - y(r, s', d))^2,$$

$$\phi_i \leftarrow \phi_i - \alpha_{critic}g_{\phi_i}$$

      Update actor by gradient decent:

$$g_{\theta_i} = \nabla_{\theta_i}\mathcal{L}(\theta_\rangle, \mathcal{B}) = \nabla_{\theta_i}\frac{1}{|\mathcal{B}|}\sum_{(s,a,r,s',d)\in\mathcal{B}}Q_{\theta_i}(s, \mu_{\theta_i}(s)),$$

$$\theta \leftarrow \phi - \alpha_{actor}g_\theta$$

      Soft update of target networks:

$$\phi_i^{target} \leftarrow (1 - \tau)\phi_i + \tau\phi_i^{target}, \quad \theta_i^{target} \leftarrow (1 - \tau)\theta_i + \tau\theta_i^{target}$$

    end for
    $s \leftarrow s'$
  end while
end for

Reward Shaping with Fields

Reward Shaping is the act of modulating the rewards returned to each team, so that the basic principles of warfare through learning, shape the decision-making of the synthetic commanders. It is a delicate endeavor as a too forced incentive will make the commanders act like too human-like, and drown out the possible hidden tactics.

Instead of trying to force a behavior, reward shaping should seek leave a trail of breadcrumbs that guide the synthetic commanders towards useful patterns. In other words, the goal is not teach a tactic, but to provide the conditions for its emergence.

Lets us reexamine the "Crossing the T" tactic from the age of sail. If a synthetic commander were to learn this tactic, two paths could be chosen.

- A system that rewards sailing in formation 1 and crossing the T of the opponent.
- A system that rewards maximizing weapon archs and hit chance, from which "Crossing the T" should emerge as the superior tactic through learning.

The second path allows synthetic commanders to discover emergent tactics by not explicitly rewarding how it does it, but instead rewarding through the *effect* the tactic has on the battlefield.

In order to encourage learning of this fashion, concepts from physics are borrowed. Since the environment is flat and continuous, ships are modeled as points, and the reward signal is a scalar, it is convenient to draw inspiration from nature and incorporate similar scalar field mechanics as found in the physical laws governing the natural world. In this way, maximization of the reward for synthetic commanders resembles the physical principle of energy minimization, by connecting the reward r with potential energy U as $r = -U$.

Gravitational Potential

From Newtonian gravity Newton [2016], it is know what masses m_1, m_2 exert a force on eachother that scales with distance D

$$F = G\frac{m_1 m_2}{D^2}$$

where the potential found by integration as (renaming r as D for consistency)

$$-\int \vec{F} \cdot d\vec{r} = U_{grav}(D) = -G\frac{m_1 m_2}{D}$$

Substituting the gravitation constant G to a suitable scaling factor R_{grav} and setting masses to 1 for each ship, I can encourage the ships to seek out

eachother by placing a similar "field" on top of each ship by designing a reward function as

$$r_i = -U_{grav}(D) = \sum_{\mathrm{j}} \frac{R_{grav}}{D_{i,j} + \epsilon} - R_{attract} D_{ij}$$

where ϵ is a small value added to avoid zero-division, i and j denote index over ships in each fleet and $D_{i,j}$ the distance between them. Experimenting with this reward function shows that the gravitational field gradient is too weak at long distances and convergence happens slowly or not at all. Therefore I add a $-R_{attract} D_{ij}$ term to the equation, in order to provide a stable gradient at longer distances. This modified gravity function will help the algorithms to convergence faster towards interesting dynamics, as they spend less time exploring far away from their opponents.

Lennard-Jones Potential

From atomic physics, the Lennard-Jones potential models interactions between atoms and molecules Lennard-Jones [1931] by combining the attractive long-range van der Waals force between atoms or molecules[3] with the repulsive short-range force of overlapping electron orbitals resulting from Pauli's exclusion principle[4]. Its generalized form is given as

$$U_{LJ}(d) = \epsilon \left[\left(\frac{\sigma}{d}\right)^n - \left(\frac{\sigma}{d}\right)^m \right]$$

where ϵ determines the depth of the potential, d is the distance between atoms or molecules, σ is where the potential crosses 0, usually interpreted as the size of the atom or molecule and n and m depend on the target particles but most used is $n, m = 12, 6$. The minimal energy configuration corresponding to an equilibrium between particles can be found by

$$\frac{dU_{LJ}(D)}{dD} = \frac{\epsilon}{\sigma} \left[-n \left(\frac{\sigma}{D}\right)^n + m \left(\frac{\sigma}{D}\right)^m \right] = 0,$$

since $\frac{\epsilon}{\sigma} \neq 0$ the solution can be found when the two inner terms cancel out

$$n \left(\frac{\sigma}{D}\right)^n = m \left(\frac{\sigma}{D}\right)^m \rightarrow \frac{n}{m} = \left(\frac{\sigma}{D}\right)^{m-n} \rightarrow \left(\frac{n}{m}\right)^{\frac{1}{m-n}} = \left(\frac{\sigma}{D}\right)$$

inverting then gives

$$\frac{D}{\sigma} = \left(\frac{n}{m}\right)^{\frac{1}{n-m}} \rightarrow D_{min} = \sigma \left(\frac{n}{m}\right)^{\frac{1}{n-m}}$$

[3] When atoms or molecules approach each other, temporary dipoles are induced from electron displacement, which result in an attractive force between them.

[4] This quantum mechanical effect arises from electrons being fermions, which prevents them from occupying the same quantum state, therefore potential energy rises rapidly and a repulsive force between them arises.

This potential will help synthetic commanders guide their ships with range superiority to find positions where they can engage the opponent, but not be hit themselves. The negative potential can on the other hand be used for ships with inferior ranges, to discourage positions where they can be hit, but seek closer should they manage to get the opponent within their range. $n, m = 12, 6$ is too strong a coupling and milder values of $n, m = 4, 2$ is used for the reward function. Using *sup* to denote the ships with superior ranges and w_{sup} representing the superior range, σ can be calculated as

$$w_{sup} = D_{min}, \qquad w_{sup}\left(\frac{n}{m}\right)^{\frac{1}{m-n}} = w_{sup}\sqrt{2} = \sigma$$

Then scaling reward with $\epsilon = R_{inf}, R_{sup}$ the reward functions become;

$$r_i^{sup} = -\sum_j R_{sup}\left[\left(\frac{\sqrt{2}w_{sup}}{D_{i,j}}\right)^4 - \left(\frac{\sqrt{2}w_{sup}}{D_{i,j}}\right)^2\right]$$

where i and j now denote index over enemy fleets with inferior or superior ranges.

Additionally, when applied between friendly ships it can reward sailing in formation but still avoid collision, reflecting the military principle of concentration of forces[5]. Using R_{form} to scale and setting the optimal formation distance of D_{form}, the reward function becomes;

$$r_i^{form} = -\sum_j R_{form}\left[\left(\frac{\sqrt{2}D_{form}}{D_{i,j}}\right)^4 - \left(\frac{\sqrt{2D_{form}}}{D_{i,j}}\right)^2\right]$$

where i, j now denote friendly ships.

Confining Boundary Potential

In quantum mechanics, a confining potential is used to model system where particles are limited to a finite region, inducing quantized energy levels of the particles trapped within. This is seen with the quantum harmonic oscillator model, which leads to the quantization of photon energy in quantized electromagnetic fields.

Inspired by this, I place the battlefield in a soft confining potential that prevents the ships from sailing too close to the boundaries, mimicking the danger of sailing close to shore. Defining the safe waters as 90% of a square

[5] A principle made famous by Carl von Clausewitz, in his book *On War* and reflected in the doctrine of all domains and warfare's in military institutions across the world Lowenthal [1977].

battlefield with length L gives the safe area of $L_{safe} = 0.9 * L$, after which the ships will be penalized with the inverse of the distance to the boundary.

$$r_i^{conf} = -R_{conf}\left(\frac{1}{\delta_i + \epsilon}\right)$$

where δ_i describes the proximity to the boundary in any direction after crossing the safe water line, and ϵ is a small number to prevent zero-division. This penalty acts a soft confinement, guiding the ships back towards the safe water in order to encourage engagements between the fleets.

Potential Field Prediction

A useful idea from electrodynamics is the Liénard-Wiechert potentials, which describes the electromagnetic fields generated by a moving particle, Griffiths [2017]. Since the speed of light is finite, the electromagnetic fields measured at a point cannot be mapped to the charged particles' instantaneous position. Instead, a retarded position is used, defined as where it was at an earlier time, allowing for the field to reach the measurer.

This causes dynamic electromagnetic field structures around moving charges and its application is not new to the military world, where the theory has been applied in radar and fire control systems.

In this case, the idea is employed in a much different way as a means to encourage interception between the ships, incorporating the anticipatory aspect of tactical thinking. Instead of calculating a retarded position, it calculates a dead-reckoning[6] position and on that is placed a predictive linear potential field ahead of the opponent ships. By making it linear I allow it to be isotropic and let the fleet compositions and the resultant rewards from the other functions determine which angle-of-attack is most suitable.

Let $\vec{d_j}$ and $\vec{v_j}$ be the position and velocity of an opponent ship to be intercepted by a friendly ship at $\vec{d_i}$, then the intercept time $t_{i,j}^I$ can be defined as the time it would take ship i to reach ship j assuming the opponent doesn't change course and speed, and the friendly ship sails at maximum speed $v_i^{\max}$:

$$t_{i,j}^I = \frac{\|\vec{d_j} - \vec{d_i}\|}{v_i^{\max} + \epsilon}$$

where ϵ again prevents zero-division. The dead-reckoning position of the opponent is given as:

$$\vec{d_j}^{\text{future}} = \vec{d_j} + \tau_{i,j}\vec{v_j}$$

On this position I place a soft attractive potential, luring ship i toward the anticipated location of ship j.

$$r_i^{\text{pred}} = -\sum_j R_{\text{pred}} \cdot \|\vec{d_j}^{\text{future}} - \vec{d_i}\|$$

[6] A navigation term for projecting the current speed and course to calculate a future position

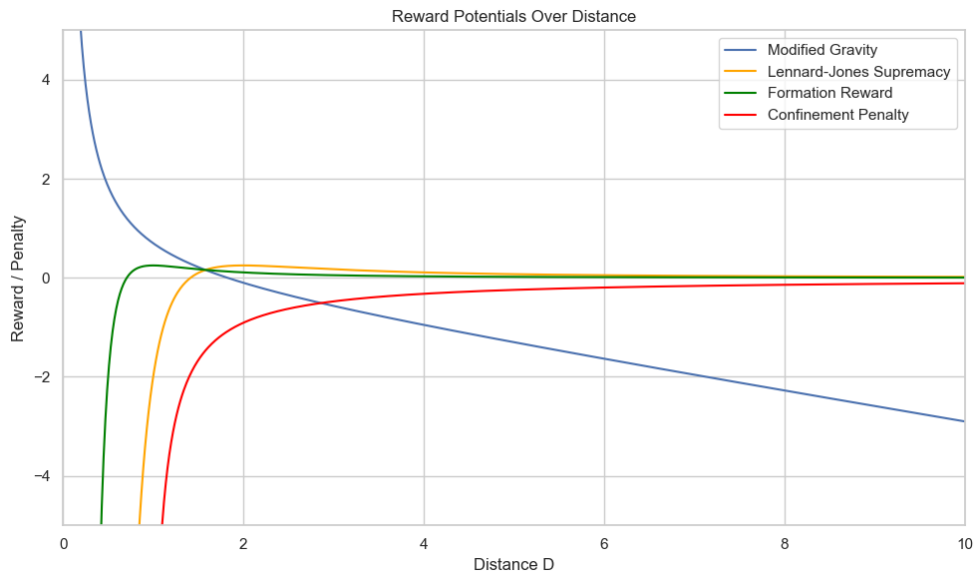


Figure 6.4: Reward potentials as function of distance. Parameters are chosen to highlight their shape. $\epsilon = 10^{-3}$, $R_{\text{grav}} = 1.0$, $R_{\text{attract}} = 0.3$, $w_{\text{sup}} = 1.0$, $D_{\text{form}} = 0.5$, $n = 4$, $m = 2$, $R_{\text{conf}} = 1.0$, $L_{\text{safe}} = 0.9$.

All the derived reward functions, can be seen in figure 6.4 and a summery of can be seen in table 6.2.

Since most of these fields would cause deterministic physics where the outcome is forced from the boundary conditions, it is not unreasonable to ask why it makes for interesting fleet engagement dynamics.

It becomes indeterminate as in addition to the fields, known reward systems known from reinforcement learning that help convergence in similar simulations are added, such as rewarding missile engagements and destruction of opponents ships. By adding a noise vector to the action-space this will allow the ships to break the normal "physical" laws in order to explore these mechanics, effectively differentiating them from the rather exotic deterministic particles they would otherwise resemble. This is analogous to quantum tunneling, where energy potentials can be penetrated despite lacking the necessary energy to classically cross them. The introduced reinforcement learning rewards and the reward scales of the fields collectively constitute the *learning signal* and its calibration is tied to the hyperparameters, which be discussed in the next section.

Mechanism	Reward Expression	Purpose
Time Penalty	$r = -k_1$	Encourage faster episodes
Hit Bonus	$r_i = k_2$ (per salvo that deals damage)	Reward successful missile engagements
Victory Bonus	$r_i \mathrel{+}= k_3$ (if ship survives)	Reward survival after enemy elimination
Gravitational Potential	$r_i = \sum_j \frac{R_{\text{grav}}}{D_{i,j}}$	Encourage movement toward enemies
LJ Superior Range	$r_i^{\text{sup}} = \sum_j R_{\text{sup}} \left[\left(\frac{\sqrt{2} w_{\text{sup}}}{D_{i,j}} \right)^4 - \left(\frac{\sqrt{2} w_{\text{sup}}}{D_{i,j}} \right)^2 \right]$	Maintain ideal distance for superior fire
Formation Cohesion	$r_i^{\text{form}} = -\sum_j R_{\text{form}} \left[\left(\frac{\sqrt{2} D_{\text{form}}}{D_{i,j}} \right)^4 - \left(\frac{\sqrt{2} D_{\text{form}}}{D_{i,j}} \right)^2 \right]$	Form tight formation, avoid collisions
Boundary Confinement	$r_i^{\text{conf}} = -R_{\text{conf}} \cdot \frac{1}{\delta_i + \epsilon}$	Discourage sailing near the edge
Predictive Intercept	$r_i^{\text{pred}} = \sum_j R_{\text{pred}} \cdot \frac{1}{\lVert \vec{d}_j^{\text{future}} - \vec{d}_i \rVert + \epsilon}$	Reward intercepting opponent's trajectory

Table 6.2: Reward terms from both scalar potentials and reinforcement learning mechanisms. k denotes constants, i indexes own ships and j indexes enemies.

Status Rewards

Each ship denoted by i carry a discrete status vector with K elements denoted by j containing the status of the ship

$$\vec{\sigma}_i = \left[f_j^{(1)}, f_j^{(2)}, \ldots, f_j^{(K)} \right]^{\mathsf{T}}$$

If there are N ships in a fleet, a concatenated fleet status vector can be constructed as

$$\vec{\sigma}_{\text{fleet}} = \begin{bmatrix} \vec{\sigma}_1 \\ \vec{\sigma}_2 \\ \vdots \\ \vec{\sigma}_N \end{bmatrix} = \begin{bmatrix} f_1^{(1)} \\ f_1^{(2)} \\ \vdots \\ f_1^{(K)} \\ f_2^{(1)} \\ \vdots \\ f_N^{(K)} \end{bmatrix}$$

By comparing the differences in the fleet status vector for each time step, a status reward can linearly be computed from a fleet-wide weight vector $\vec{w}_{fleet}$ as

$$R_{status} = \vec{w}_{\text{fleet}}^{\mathsf{T}} \left| \vec{\sigma}_{\text{fleet},t} - \vec{\sigma}_{\text{fleet},t-1} \right|$$

Hyperparameter Setting

Hyperparameters determine how well the synthetic commanders converge towards a single offensive/defensive tactic. Typically for deep deterministic policy gradient algorithm, the set of hyperparameters where stable learning happens is as thin as a knife edge and is hard to find with an advanced prebuilt environment.

Therefore, building the environment can be a tedious task, as adding features or modifying rewards can change the necessary hyperparameters needed for proper convergence. The methodology used is therefore to carefully achieve stable learning with observed expected result for each feature added to the environment, and then add them one-by-one, recalibrating hyperparameters and learning signal as progression happens.

A classic problem in actor-critic methods (which I have become very familiar with) is *policy collapse*. It happens when the critic overestimates the action value $Q(s|a)$, and inadvertently guides the actor to pick actions that are not reflective of the best long term action-value. Since the critic help determine the scale of the gradient decent step, such overestimations can cause overly confident updates, converging toward inferior actions. This is especially pronounced when meeting discontinuities.

Noise Exploration

Noise is injected into the action space in order to encourage exploration. Three interchangeable noise shapes was explored.

- **Gaussian Noise**, simple familiar noise shape

$$a_t \leftarrow a_t + \mathcal{N}(0, \sigma^2 I)$$

- **Ornstein-Uhlenbeck Noise**, known from the physical model of a massive brownian particle under the influence of friction, it carries temporally correlated perturbations giving "momentum" to the noise injected. This is useful for a wider exploration, given its greater ability to penetrate the reward valleys in order to escape local suboptimal maxima.

$$a_t \leftarrow a_t + \nu_t, \quad \text{where} \quad \nu_{t+1} = \nu_t + \beta(\mu - \nu_t) + \sigma\mathcal{N}(0, \sigma^2 I)$$

  where β is the decay rate, μ is the mean reversion target, and σ controls volatility similary to the gaussian $\sigma.\nu$ acts as a [7]

- **ε-Greedy Noise**, popular in reinforcement learning problems, every action has a probability ϵ of choosing a uniformly random action across the action space.

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \varepsilon \\ \max Q(s_t, a), & \text{with probability } 1 - \varepsilon \end{cases}$$

A visualization of the noise shapes used can be seen in figure 6.5.

A problem with Gaussian noise became apparent when observing the agents primarily sailing eastward. As courses component of the action space is defined as values between $[-1, 1]$, which then gets projected to $[-\pi, \pi]$, a bias is introduced in the beginning of training as the action component is centered around 0. This corresponds to eastward maneuvres as 0 is projected to 90 deg. A similar but less pronounced bias was seen with Ornstein Uhlenbeck noise if the damping term was set too high. Therefore, I settled on using the ϵ-greedy noise for all experiments.

All the noises are nested in decay functions, in order to anneal the correct behavior during training. For this project I explore linear and exponential decay as;

- **Linear Decay**

$$\eta_t = \eta_0 - \left(\frac{\eta_0 - \eta_T}{T}\right) \cdot t, \quad \text{for } t \leq T$$

  and clamped as:

$$\eta_t = \max(\eta_t, \eta_T)$$

[7] Since the noise is applied per step, the usual Δt term is set to 1.

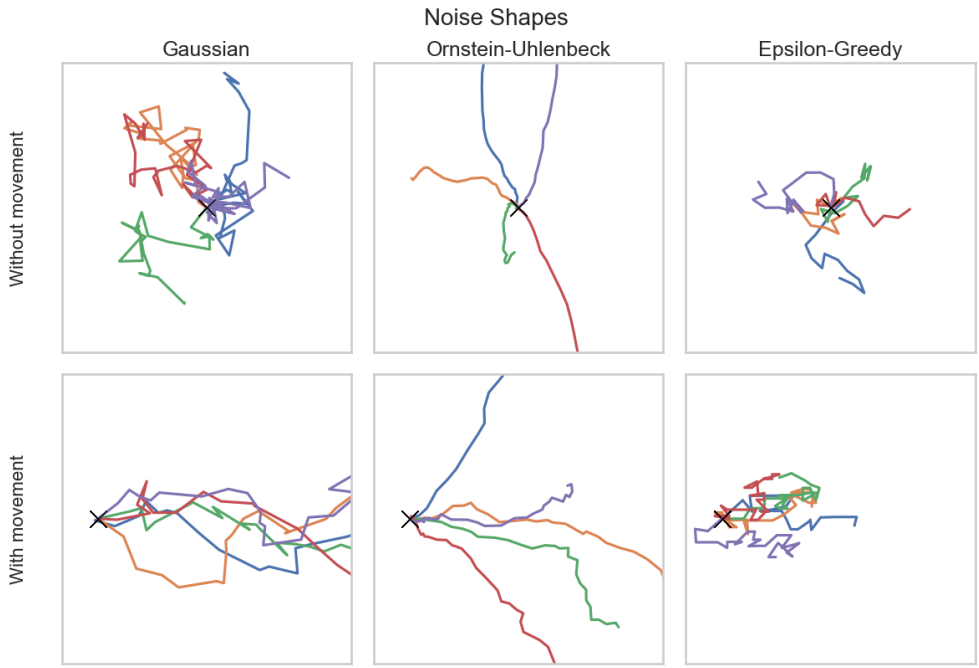


Figure 6.5: The different noise shapes used throughout the project. In color are the five sample trajectories a ship takes under the influence of noise from the starting point marked with a red X. Upper row is the noise applied to a ship dead in water, as in the initial conditions. Lower row is the noise applied to a ship sailing eastward, as when the policy has settles directionally in the local region. Used parameters are;
$t = 50, \quad \sigma = 2, \quad \theta = 0.5, \quad \mu = 0, \quad \epsilon = 0.5$

- **Exponential Decay**

$$\eta_t = \eta_0 \cdot \exp(-kt)$$

where η_0 is the initial noise level, $k > 0$ is the decay rate and k is chosen such that σ_t reaches a target value η_T at step T, by solving:

$$k = \frac{1}{T} \ln\left(\frac{\eta_0}{\eta_T}\right)$$

It should be noted that other more advanced noise schemes could be interesting to explore, in order to inject noise for exploration at different times during the training. Examples include; cyclical noise, entropy-based by measuring KL-divergence between policies and state-based noise where the magnitude is determined as a function of agent confidence in its current state.

However, these are beyond the scope of this project.

Learning Rates

A warmup phase is necessary to fill up $\mathcal{D}$ with sufficient transitions, in order to have a decent sampling pool to draw from. Otherwise the sparse initial data will cause biases that eventually blow up, causing instability and eventually policy collapse.

At first, I limit the commanders to one ship each with 1 hitpoint each and enable autofiring with probability of hit at $p_{hit} = 1$ when within range. The effective action space then becomes the 4-dimensional

$$[\text{course}_{alice},\ \text{speed}_{alice},\ \text{course}_{bob},\ \text{speed}_{bob}]$$

The reward function is defined as the gravity term from last section with a termination reward for the destruction of the opponent

$$r_i = \frac{R_{grav}}{D_{ij} + \epsilon} - R_{attract} D_{ij} + R_{hit} e_{ij}(t)$$

with coefficients set at $R_{grav} = 0.1, \quad R_{attract} = 3, \quad R_{hit} = 5$. A simple noise exploration scheme is setup with a ϵ-greedy noise with $\epsilon = 0.2$, A gridsearch is then performed over hyperparameters $\alpha_{critic}, \quad \alpha_{actor}$ and τ to get a good starting point as seen in figure 6.3. A good set of parameters is evaluated based on 3 metrics;

- **Critic loss**

  $$L_{\text{critic}} = \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \big(Q_\phi(s,a) - y(r,s',d)\big)^2$$

  Which measures how well the critic approximates the true action-value function. By adjusting α_{critic} and τ it should converge toward a low positive value, signaling that the target action-value function is getting captured. Oscillations may happen but should dampen out through training. Divergent oscillations suggest failure to capture the true action-value indicating policy collapse.

- **Actor loss**

  $$L_{\text{actor}} = -\frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} Q_\phi\big(s, \mu_\theta(s)\big)$$

  Which is dependent on the critic loss and should converge towards a stable value. Oscillations suggest that the critic is not guiding the actor correctly, which happens if the gradient reward gradient is too flat or if the actor's learning rate is faster than the critic. Divergence signals that the critic overestimates the action value and keeps wrongfully training the actor, indicative of policy collapse.

- **Visual Inspection** Visual inspection of trained trajectories in order to determine if commanders pursue realistic tactics and are not performing reward hacking, in this case in the form of exploitation of wall penalties or convergence toward orbital dynamics[8].

In figure 6.6 the results can be seen and found starting parameters can be seen in table 6.3;

The starting parameters get the learning going, but in order to allow the algorithm to fine-tune its approximations, an exponential decaying learning rate schedule is introduced as

$$\alpha(\mathrm{n}) = \alpha_0 \exp\left(-\frac{\ln(100)}{n_{\max} \cdot n}\right)$$

which reduces to 1/100 of the original value α_0 at step $n = n_{max}$ after warm-up.

Category	**Parameter**	**Value**
Environment	Grid size (L)	100×100 km
	Episode length ($T_{\max}$)	200 steps = 3 hours, 20 min
	Ships per team (N_{team})	1
	dt	1min
	Total runtime	200000 steps
Action bounds	v (speed)	$[0,\ 20\text{ kn } \approx 37\text{ km/t}]$
	h (course)	$[-\pi, \pi]$
DDPG	Network depth (D)	3
	Hidden layer width (W)	128
	Actor learning rate (α_{actor})	1×10^{-5}
	Critic learning rate (α_{critic})	1×10^{-4}
	Discount factor (γ)	0.95
	Polyak coefficient (τ)	0.01
	Batch size (b)	64
	Replay-buffer size	1×10^5 transitions
	Action noise (Gaussian decay)	$\sigma_0 = 1, \eta = 0.995$

Table 6.3: Initial hyper-parameters

[8]For this setup, if the discount factor γ is not limited and in balance with the victory bonus, the agents may discover is it more rewarding to avoid destroying each other and just orbit outside the weapon engagement range accumulating rewards until max steps are reached.

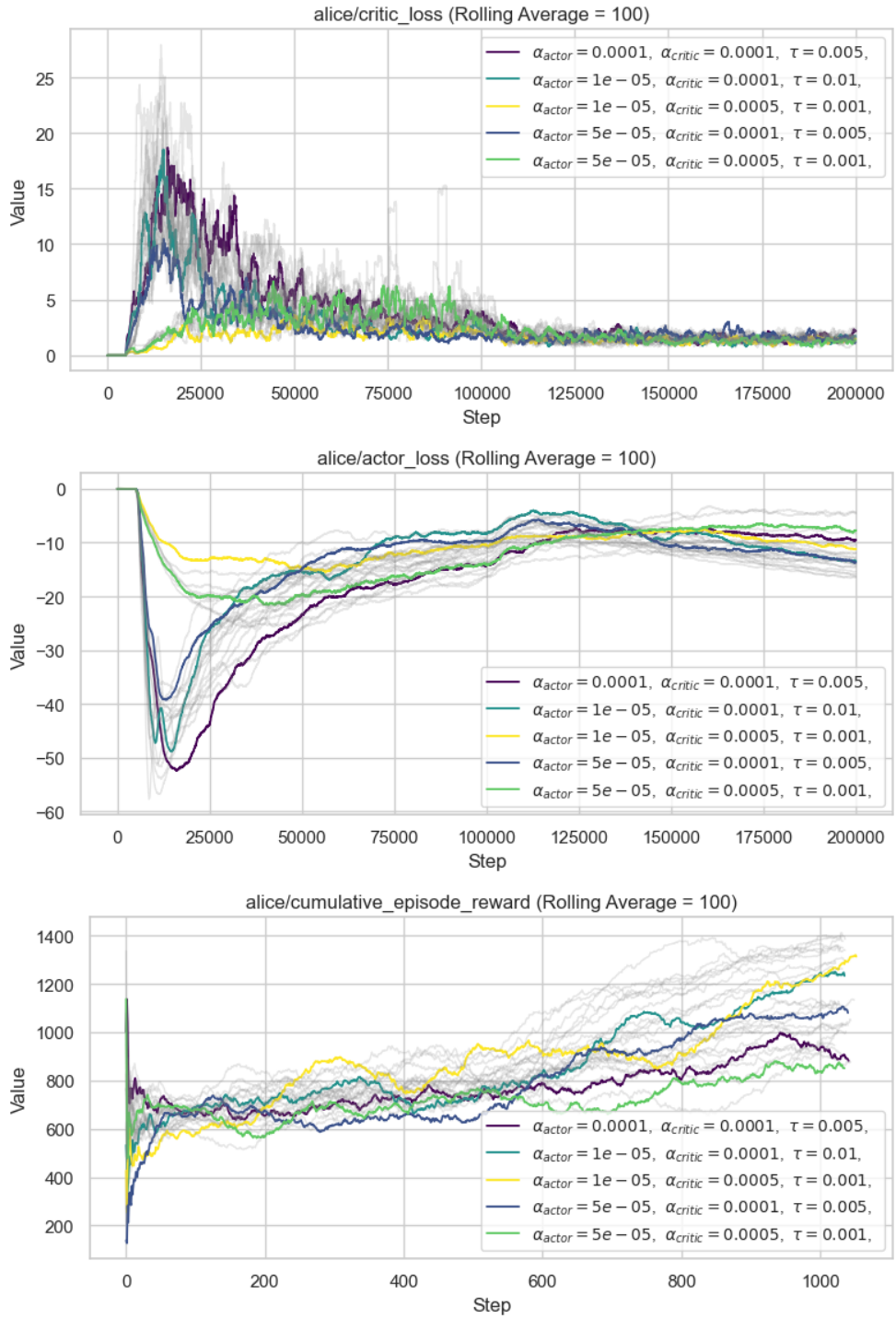


Figure 6.6: Rolling average of 100 for loss and reward plots for a simple adversarial setup, $\gamma = 0.95$ and batch size set at 64. A total of 27 runs was performed and 5 are highlighted, the rest are visible in opaque in the background.

Stability Enhancements

In addition to the target networks and the warmup phase, several tricks are applied in order to stabilize learning and minimize the risk of policy collapse.

Twin Delayed

Taking inspiration from the Twin Delayed Deep Deterministic Policy Gradient algorithm, Fujimoto et al. [2018], I introduce some changes to the original algorithm. First, for each agent two sets of critics and target critic neural networks are initialized. Both are asked to evaluate the state-action value and the most conservative of them are then selected as the target, as

$$Q_{\text{target}} = r + \gamma(1-d) \cdot \min\left(Q_1'(s', a'),\ Q_2'(s', a')\right)$$

Secondly, in order to prevent the actor from overtraining on a sharp peaks in the state-action function, noise is injecting to the actor target policy as

$$a' = \pi'(s') + \epsilon, \quad \epsilon \sim \text{clip}\left(\mathcal{N}(0, \sigma^2), -a_{low}, a_{high}\right)$$

where it is clipped between a_{low} and a_{high}. This smooths out the state-action estimates over the adjacent actions, resulting in easier-to-capture gradients.

Thirdly, a delayed update is added to the actor, in order to secure the critic is accurate and robust before updating, avoiding training of the actor on unstable state-action estimates. It is implemented such that the critics update every step, but the actor only updates every 2 steps, as recommended in the paper.

These methods are described in detail at OpenAI, noa.

Environment Compatibility

In the previous section, I discussed the challenges faced by optimization in hybrid spaces for warfare simulations, and I wish to elaborate on how the adversarial deep deterministic policy gradient setup deals with these.

1. **Trajectory optimization:** Deep deterministic policy gradient is the natural extension of the deep Q-learning network into continuous spaces. It is by design capable of continuous input and output. By choosing a suitable low time-step dt, the resultant trajectory will smooth out and resemble numerical integration methods from known physical simulations. The role of the target networks is both to stabilize learning, but in effect also averages out the initial noise and randomness introduced by exploration[9]. The resultant optimized policy should therefore resemble

[9] In a similar fashion to how multiple GPS positions would give a distorted picture of velocity, but through averaging can provide an accurate vector

a nearly-continuous line representing the maneuver component of the tactic.

2. **Opponent Modeling and Counterplay:** Each time-step both agents make actions on the environment and therefore, effectively change the environment for the opponent. In this fashion, counterplay is introduced into the simulation.

3. **Information Uncertainty and Hidden Tactics:** Despite the algorithm being called deterministic, the stochatic element necessary for evaluating tactics is introduced with both the noise added to the action-value function and also inherently in the environment by modeling engagements success with a probability. By limiting the returned state vector provided to each agent and restricting the information, it should be possible to introduce the so-called "fog of war" into the naval simulation.

4. **Abstract Asymmetric Objective Optimization:** The reward returned for each agent can be computed differently and reflect the asymmetric objectives each commander has to achieve, as I will explore in the next chapter.

5. **Computational feasibility:** Through the memory buffer, the amount of environment interactions can be reduced. The actual cost of computation will still need to be measured, as it will impact how such a tool is implemented in real life, as I will discuss in later chapters.

Chapter 7

Results

In this chapter, I present the results from running the adversarial commander setup on a few tactical scenarios with the goal of demonstrating its general capabilities. As mentioned in chapter 5, a tactic is a set of actions, chosen to with intent in order to navigate the state-space while anticipating the opponents actions. I will explore tactics in the state-space, under the assumption that both commanders seek optimized tactics.

As I will return to later in the chapter, I was unable to get the algorithm to work in hybrid spaces, and therefore the exploration of the tactical space solely focuses on the maneuver component of tactics.

Defining a Tactical Scenario

Recalling the tactic definition as

$$\{F_{nested}(S_{start}, a_{sequence})\} =$$

$$\{F(F...(F(s, a_1)..., a_2), a_i)| \text{ for all } s \in S_{start}\} \rightarrow S_{end}$$

$$\text{where } V(S_{start}) < V(S_{end}), \quad \{S_{start}, S_{end}\} \subseteq \mathbf{S}$$

The transition function now is the key linking states together with actions, which means adjacent states are within each others

In the context of the adversarial commander model, the estimated evaluation function is given as the cumulative reward attained during a trajectory $V(s) = r_{cum}$. The set of starting states S_{start} can be either a single fixed set of ship positions or a range of them, simulated by randomizing their start positions in a given region. The end states S_{end} correspond to the termination states mentioned in previous chapter and can vary given the scenario under analysis.

So for a tactical scenario and its corresponding tactical space that is to be explored, one needs to define;

- **S**: What is the state-space?

- a: What is the action space?
- F_{trans}: What are the rules/physics of the simulation?
- S_{start}: What are the starting states?
- S_{end}: What are the termination criteria of the scenario?
- $V(s)$: How are the rewards shaped?

From these an optimization process happens through the adversarial commander setup, which produces the most optimized offensive/defensive tactic for both commanders, given they both act with perfect reasoning.

Simple Attraction

The simplest simulation that can be run, is one where each fleet consist of a single ship and the goal of both is the annihilation of the opponent. The expected behavior is that they attract, seek out eachother and shoot at the same time, both achieving their goals.

The scenario is defined as;

- **S**: $[\mathrm{x}, \mathrm{y}] \times 2$, where $x, y \in [-100\mathrm{km} : -100\mathrm{km}]$.
  For simplicity, Alice and Bob has $\mathrm{hp}_{\max} = 1$. Ammunition is unlimited.
- a: $[\mathrm{course}, \mathrm{speed}] \times$ number of ships, for each commander.
  where course $\in [-\pi, \pi]$, speed $\in [0, 20\ \mathrm{kn}]$
- F_{trans}: Autofiring enabled for Alice's and Bobs ships when within weapon range. Weapon ranges are set to $w_{range} = 10\mathrm{km}$ and probability of hit at $p_{hit} = 1$.
- S_{start}: For Alice, her ship is placed in $[-80\mathrm{km} : -80\mathrm{km}]$ and for Bob, his ship is placed in $[80\mathrm{km} : 80\mathrm{km}]$.
- S_{end}: States corresponding to the annihilation of Alice and/or Bobs ships.
- $V(s)$: Only the modified gravity is enabled $R_{grav} = 0.1$, $R_{attract} = 1$. $\vec{w}_{\mathrm{fleet}}$ is defined such, that the annihilation of the opponent rewards $+10$ and exceeding maximum steps punishes with -10.

The resultant trajectory evolution can be seen in figure 7.1. In the beginning, the trajectory is heavily guided by noise but later the agents actions take over. Midway in green, the agents seem to converge on expected direct trajectories. Later in red and purple the agents seems to be approaching a unexpected behavior, which signals that reward hacking is occurring.

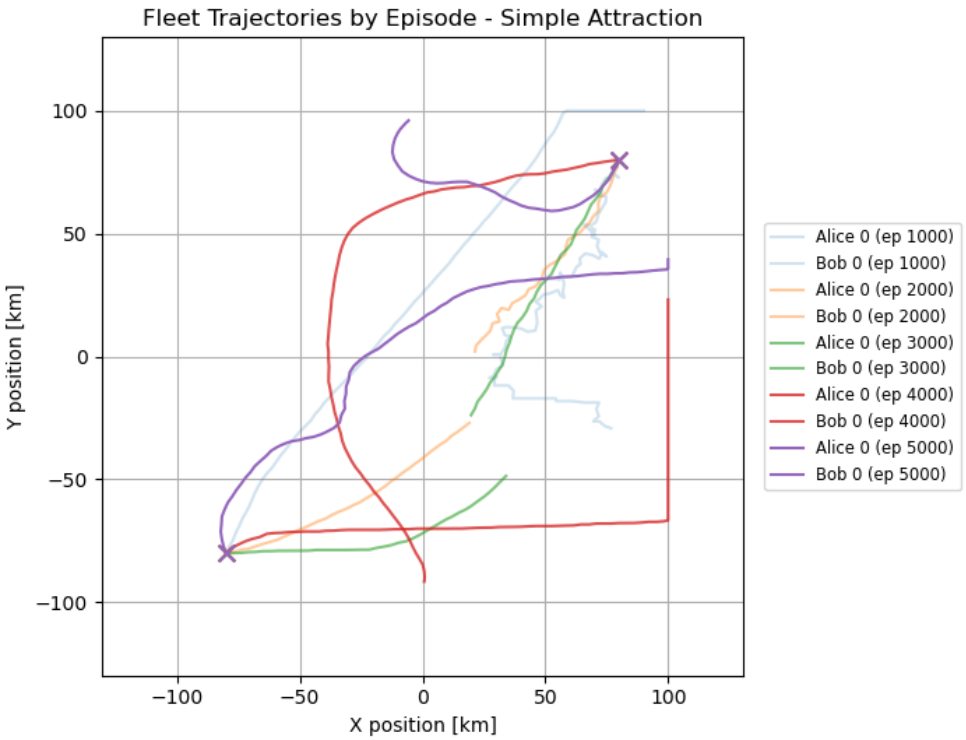


Figure 7.1: Trajectory evolution from running the simple attraction setup. Starting position marked with a X.

This highlights a consideration to be taken in symmetric scenarios, as hysteresis phenomena appear when the only asymmetry in the setup is the noise. As noise decreases and the scenario becomes increasingly symmetric, this behavior appears. I will return to this in the discussion.

Cat and Mouse

Next experiment to conduct is a simple cat and mouse scenario, where one fleet is superior to another and whose objective is the annihilation of the enemy fleet. The opposing fleet has the objective of staying alive as long as possible.

Defining the scenario as;

- **S**: $[\mathrm{x}, \mathrm{y}] \times$ number of ships, where $x, y \in [-100\mathrm{km} : -100\mathrm{km}]$.
  For simplicity, Alice has unlimited ammunition and HP, and Bob has $\mathrm{hp}_{\mathrm{max}} = 1$.

- a: $[\mathrm{course}, \mathrm{speed}] \times$ number of ships, for each commander.
  where course $\in [-\pi, \pi]$, speed $\in [0, \mathrm{max\ speed}]$

- F_{trans}: Autofiring enabled for Alice's fleet when within weapon range, disabled for Bob. Weapon ranges are set to $w_{range} = 10$km and probability of hit at $p_{hit} = 1$.
- S_{start}: For Alice, her ships are placed in a circle around $[-80\text{km} : -80\text{km}]$ with radius 5km and for Bob, his ships placed in a circle around $[80\text{km} : 80\text{km}]$ with radius 5km.
- S_{end}: States corresponding to the annihilation of Bobs ships (Alice win condition), or 100 steps limit reached (Bob win condition).
- $V(s)$: $R_{grav} = 0.1$, $R_{attract} = 1$, $R_{form} = 0.2$, $R_{sup} = 0.5$, $D_{form} = 0.1$. $\vec{w}_{\text{fleet}}$ is defined such, that the annihilation of the opponent rewards $+10$ and exceeding maximum steps punishes with -10. Bob gets a $+10$ reward for staying alive until max steps.

Starting off, I commit 1 ship to each fleet. The goal is to find the most effective tactic for each fleet, by running the adversarial commander scheme from last chapter.

The trajectory evolution can be seen in figure 7.2, which shows that the initial reaction from both agents in the early training is as expected. Alice seeks out Bobs position, and Bob attempts to run away. As training evolution progresses, circular hysteresis behavior emerges.

I then added a second ship in Alices fleet, and ran the experiment. Unfortunately Alice does not seem to be able to form any meaningful coupling between the state-space and the actions of the second ship.

Distinguishing Separate Tactics

When more than one good tactic arises from a given situation, a careful approach must be taken. Take, for example; Alice has two minor important ships of equal value, and in order to win, she has to sacrify one of them. In reinforcement learning terms, this means that multiple maxima was found. A proposed method for distinguishing them from each other is to produce several optimized trajectories and collect them in a set. An unsupervised learning scheme like UMAP can then be applied to the trajectories to see which groups are and which are distinct. The issue of different trajectory lengths can be solved by fixing the positions after S_{end} has been reached, until max steps has been reached. Unfortunately, I did not manage to get enough tactics from a given scenario to implement this.

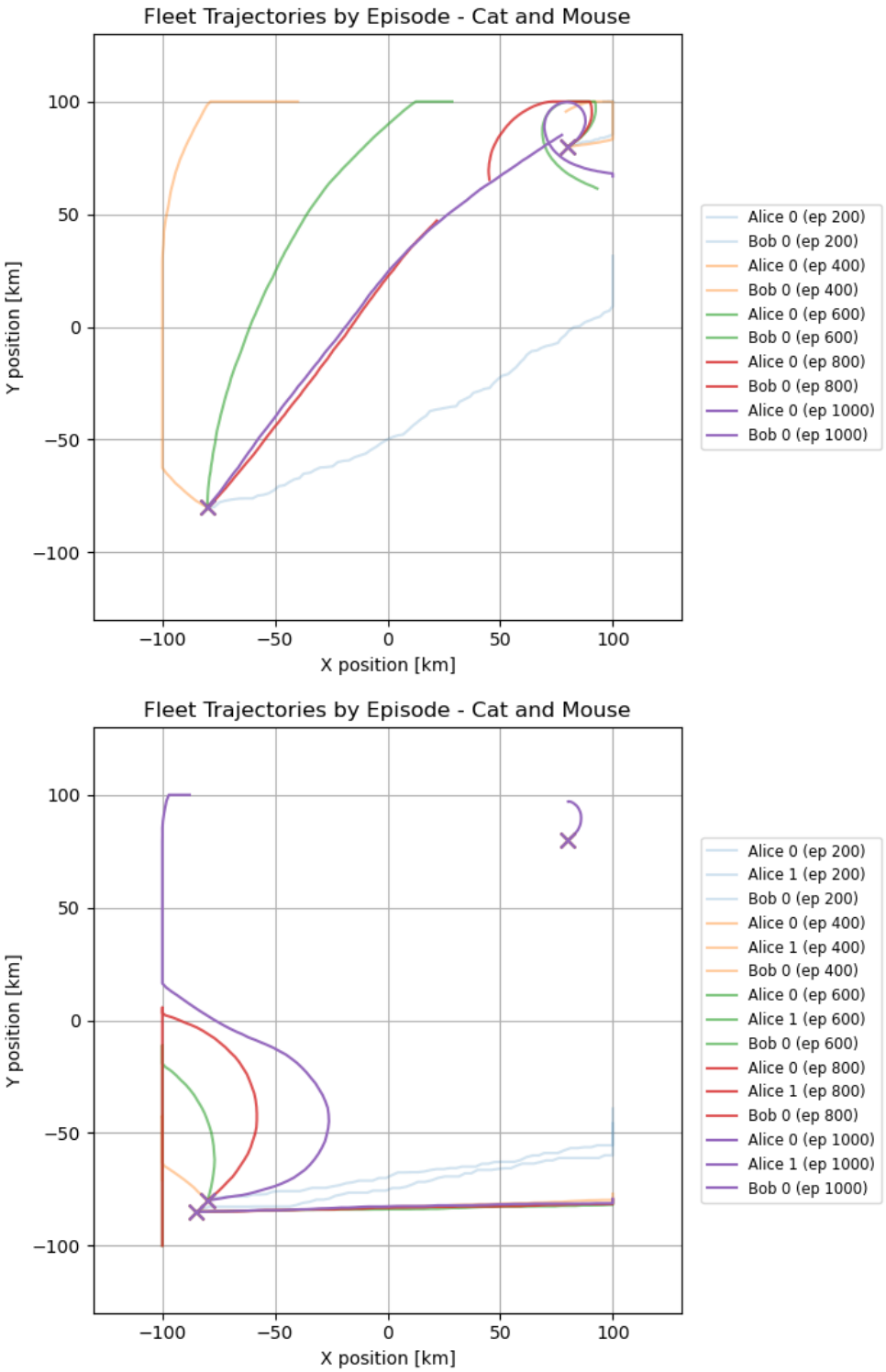


Figure 7.2: Trajectory evolution for the cat and mouse setup. Alice has to hunt down Bob, and Bob has to flee. Top: 1 vs 1 ship setup, circular reward hacking behavior becomes emergent as they train against each other. Bottom: 2 vs 1 ship setup, Bob remains static in training evolution and Alice second ship doesnt seem to be learning well.

Chapter 8

Discussion

In this chapter I will discuss the result presented in the last chapter, elaborate on problems faced and finally take a look towards a future implementation of the adversarial approach to tactical optimization.

First I will return to the characteristic of a naval simulator and consider how well I managed to achieve the goals.

1. **Trajectory optimization:** The adversarial setup does indeed produce nearly continuous courses of action, reminiscent of numerical methods from simulation. Unfortunately, I was unable to consistently produce stable training evolutions, and the algorithms kept jumping from one maxima to another as the agents kept taking into account the opponents policy.

2. **Opponent Modeling and Counterplay:** As seen in the simple attraction scenario in figure 7.1, it is apparent that the agents policies influence each other. First, they converge toward an optimal tactic sailing straight toward each other, but later hysteresis-like phenomena seem to emerge, where they repeatedly overly perform counterplay instead of stable learning.

3. **Information Uncertainty and Hidden Tactics:** Hidden tactics through restricted state-vectors were ultimately beyond the scope of the project and initial trials showed ridiculous reward hacking.

4. **Abstract Asymmetric Objective Optimization:** Asymmetric objectives through asymmetric reward shaping, were not only an achievable goal, but seemed to improve learning when implemented.

5. **Computational feasibility:** The computational time for each scenario was on the order of 1-2 hours running on a laptop 2th Gen Intel(R) Core(TM) i5-12500H, 3.10 GHz CPU. Training time could be reduced by parallelizing the training function each time step and further decreased

by experimenting with larger batch sizes. The complexity of tried experiments was ultimately rather low, and given the state-explosion problem it is likely to increase dramatically for more complex problems. Applications to real scenarios would require much greater complexity, and it is unlikely to run these on existing desktop machinery onboard warships. Instead, dedicated modular setups onboard larger vessel or through data centers inland would be likely be required.

Suboptimal Emergent Policies

First, it is evident that finding the right balance between hyper-parameters, the complexity of the environment and agents, and reward shaping, is no easy feat. Small changes to either can throw the algorithms off balance and cause often hard-to-see failed policies that optimize towards undesired goals.

Four possible causes of the problematic policy evolution are identified:

- Inability to handle near-discontinuities with the deep deterministic gradient policy. The gradients of the reward functions might be too steep for the algorithms to handle.
- Choice of hyperparameters might be wrong for this specific type of environment and the gridsearch might not have been wide enough.
- The energy potential approach might not provide the necessary reward shape for guiding the agents toward desired tactics.
- Completely decoupled agents might always exhibit policy hacking when acting on the same environment.

In chapter 10, I will provide guidance for anyone wishing to continue the research and what how I would try next in order to deal with these problems had I more time.

Potential Future Applications

This fine-tuning for every scenario makes the dream of a generalized model easy to apply to many different scenarios unlikely with the current development of tried reinforcement learning algorithms. Given how fast the field is developing, I predict such stable algorithms will arrive in time, which opens for a thought about how these tactical optimization methods could be used in future military planning.

Military decision-making is often divided into three different levels of analysis, namely the tactical, operational and strategic levels, and they provides a useful model for discussion about the implementation of an adversarial optimization scheme Sakala [1997]. They exist in all domains of warfare, but here I will summarize them in context of the naval domain.

1. **The Tactical Level**: On the tactical level we find the engagement of ships or fleets. It concerns specific tactics employed by either side, like pincer movements, deployment of helicopter assists, division of ship-level roles and use of forward looking pickets.
   Application: The required speed of decision-making necessary in Air Defense[1] forces military tacticians to think outside-the-box. The time from detection to impact can for modern missiles be as low as 20s which combined with , where crews with both proper preplans and good training level will quickly become saturated. A pretrained model could help anneal an optimized plan which that could be initialized with a single click. Many facets of protecting high-valuable units from threats could also be optimized by running simulations on scenarios. By repeatedly testing which formation structure best ensure survival, dispositioning of ships would not be solely based on experiences from likely outdated doctrine from a previous technological generation but reflect the true weaknesses and strengths of the weapon systems. The new generation of autonomous weapon systems such as drones, could benefit from being trained on scenarios they would face if deployed. In that way they would be able to react to changes even when jammed or maintaining a non-emission profile. Are multiple systems deployed, the use becomes more apparent, as coordination between several systems could be pretrained to counter defense system and deliver ordinance in a effective fashion.

2. **The Operational Level**: On the operational level we take a step back and look at the general area of conflict. It is a limited theater of war and for example could be the Baltic or Black Sea. The operational planners on this level rarely micro-manage their ships but instead give general instructions for the division or fleet commanders to follow in term of effects on the battlefield, coordinating the effort of many subgroup towards a common goal. For example an operational goal could be the amphibious landing of landforces, which could require coordination between a fleet battlegroup, minesweeper forces, the transport ship and submarine assists.
   Application: By varying the amount of ships required to achieve a given goal, a critical tipping point can be found and give a baseline for commanders on how many ships would be required to safely execute a task. Additionally, preplans could be designed around simulated threats such that operation wide weaknesses are known and countered.

3. **The Strategic Level**: The strategic level is the highest level of military decision-making and it concerns with the overarching direction of a conflict. On here we find not only high-ranking military personel, but also statesmen of political power. Coordination between nations, alignment

[1] The doctrinal group for the act of defending a ship or fleet against incoming air threats.

of common objectives and decisions about starting or joining a conflict appear on this level.

Application: If unilaterally implemented, simulations could act as a deterrent, if both sides have quantitative knowledge about the best outcome of engagements. If there is a calculated million-to-one chance of success, it would hard to argue for its execution. By running simulations on known threats and possible counters, it would be possible to estimate the effect weapon systems and ship platforms would have in specific scenarios. This could help guide acquisition departments in military institutions as to what systems to acquire and in what numbers, in order to accomplish a given military goal.

Chapter 9

Conclusion

In this chapter I summarize key findings

Mathematical Foundation of Tactic

Taking inspiration from Chess and how that game has changed since the introduction of chess computers, I laid the mathematical foundation for what a tactic is from a game theoretic perspective. I've categorized tactics into three groups relating to their use by humansm but unfortunately I was unable to explore their use in relation to the developed tactics.

Naval Environment

I've build a navel environment with the field-standard gymnasium python packages, and described how to expand this with more features. The naval environment is in hybrid spaces and I've attempted to deal with the challenges that arise from that through biased sampling and priming the agents memory buffer with synthetic transitions. Unfortunately I could not get it to work.

Reward Shaping through Energy Potentials

I've designed a energy potential approach to reward shaping, which mimics the way nature seeks energy minimas. Introduced potentials was a gravity term, a Lennard Jones potential shape, a confinement potential and a predictive field.

Adversarial Commanders

While I was unable to get adequate results from the adversarial setup, the algorithm does work when one opponent is static, as seen in 6.2. The results from the adversarial setup suggests that learning becomes unstable as the agents "overtrain" on eachother, causing a suboptimal behavior in their resultant tactic.

Chapter 10

Challenges

At mentioned, I did many attempts at getting the algorithms to work in hybrid spaces, here is a short description of my attempt.

Attempts at Dealing with Hybrid State-spaces

As discussed in chapter 4, the ideal naval warfare simulator environment is in hybrid space, with both discrete and continuous state-space and action-space. Initially problematic for the strictly continuous action space and state space of the deep deterministic policy gradient algorithm. I will here map the attempts at dealing with these. The spaces can be defined as

$$\mathbf{S} = \mathbf{S}_{continuous} \times \mathbf{S}_{discrete}, \quad \mathbf{A} = \mathbf{A}_{continuous} \times \mathbf{A}_{discrete}$$

Where the dimensionality of $\mathbf{S}$ and $\mathbf{A}$ depends on how many features and how complex the simulator is. By noticing that the discrete components of $\mathbf{S}$ can simply be understood as a continuous number, by computationally changing it to a float and that $\mathbf{A}$ can be modeled as continuous actions being transformed into discrete by defining a round function;

$$\mathbf{round}(x) = \lfloor x + 0.5 \rfloor$$

$$\mathbf{A}_{discrete} \rightarrow \mathbf{round}(\mathbf{A}_{continuous})$$

Firing Mechanics

Next was to expand the action space by adding firing mechanics as

$$[\text{course}_i,\ \text{speed}_i,\ \text{salvo size}_i,\ \text{target}_i], \quad i : \text{index over all ships}$$

where salvo size $\in [0, 1]$ and target $\in [-1, 1]$. Each ship is given a stockpile of missiles at the start of each episode, with a weapon range and

The firing process begins as

- The salvo size action component is produced by the actor as a continuous value. It is then converted to a discrete number of missiles to fire by multiplying with a maximum salvo size and rounding.

$$\text{missiles to fire} = \text{round}(\text{salvo size} \cdot \text{max salvo size})$$

  A minimum firing threshold is introduced as Firing : Missiles to fire $> F_{threshold}$ with $F_{threshold} = 0.5$ allowing commanders to conserve their ammunition.

- The target action component gives a continuous value that is mapped into the opponents ships j as

$$j = \left\lfloor \frac{\text{target} + 1}{2} (N_{\text{opp}} - 1) \right\rfloor,$$

This design should allow agents to capture the firing dynamics while keeping the action space limited. The noise added at the beginning to the action space is a modified $\epsilon -$ greedy variant. At every step, with probability $1 - \epsilon$ the action space is spiked with a firing action noise on the salvo size and target parameter, such that it is provoking firing transitions which help the commanders identify when and where it can shoot.

Over training, the rate at which this occurs is linearly reduced as

$$p_{\text{fire}}(t) = \max\left(p_{\text{final}},\ p_0 \cdot \left(1 - \frac{t}{T_{\text{decay}}}\right)\right)$$

where

- p_0 is the initial fire probability (e.g., 0.30)
- p_{final} is the final fire probability (e.g., 0.05)
- T_{decay} is the number of episodes over which annealing occurs
- t is the current episode index

Biased Sampling

A problem occurs if the algorithm is executed and sampling from the memory buffer is equal among all transitions. For a 1 vs 1 ship scenario with $p_{hit} = 1$ a shooting event happens maximally one pr episode, and in the early training early not at all. Experiments show that 5 shooting events happen in first 100 episodes. If an early stop is activated at 100 steps, this means that the memory buffer will fill with ≈ 10.000 transitions, of which only 5 will contain information regarding the firing conditions.

These shooting events are thus very rare, and that should be reflected in the training of the agents, as to not drown out the importance of firing mechanics by overly training on movement transitions.

A way to accomplish this is biased sampling. By collecting shooting transitions in a separate memory buffer, here called the rare event memory buffer, I can force the critic to sample rare transitions with a rare event rate compared to the main memory buffer. By forcing the critic to sample a ratio of samples from the rare and main memory buffer, the rare transitions will be presented more often to the critic, and the critic will learn the mapping between firing actions and firing rewards quicker and will not overly focus on movement.

Unfortunately, there was never observed a working coupling between states and actions, and at best the shooting seemed to be either random or always activated. It is likely that the discontinuities introduced by the discrete transformation causes instability in the critics estimate of the action-value function.

Chapter 11

Future Research

As a guide to future researchers, I've dedicated a chapter for my thoughts on how to deal with the problems stated in chapter 7.

Sharp Gradients

One of the possible causes of the unstable policy evolution could lie in the potential energy based approach. A different approach rewarding basic military principles through selected parameters is an interesting concept. Examples could be;

- Rewards based on course and speed incentivizing head-on maneuvers akin to the former British Navy approach of always closing distance.
- Punishing idle behavior incentivizing seizing initiative.
- Rewarding sailing in known formation shapes.

Advanced Learning Schemes

The policy evolution seems to spin out of control when both agents are training on each other simultaneously. By designing an advanced learning scheme where the agents take turns improving their policy, this might be averted. An example could be Alice trains for 100 episodes, then Bob trains for 100 episode, and so on. By locking the training of one agent, the other is able to train in isolation, while still taking into account the opponent. This might provide the necessary stability that is needed to avoid suboptimal behavior.

Different Framework

MADDPG has proven useful for cooperative training of agents using a centralized critic. The idea was not explored here, as I wanted to have true competition between agents, and the centralized critic was expected to introduce bias by maximizing the total rewards by aligning itself with one actor

and neglecting the other. If its possible to reframe how tactics are defined, in such a way that a centralized critic can optimize on both actors without introducing bias, this might make a sound framework for further research.

Bibliography

Royal Danish Navy. Søværnet, February 2025. URL https://www.forsvaret.dk/da/organisation/soevaernet/.

Alfred Thayer Mahan. *The Influence of Sea Power upon History, 1660-1783*. Little, Brown and Company, 1890.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm, 2017. URL https://arxiv.org/abs/1712.01815. Version Number: 1.

Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander S. Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom L. Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, November 2019. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-019-1724-z. URL https://www.nature.com/articles/s41586-019-1724-z.

CMO. Wargaming. Technical report, Forsvarsakademiet, Center for Maritime Operationer, 2021. URL https://www.fak.dk/globalassets/fak/dokumenter/publikationer/-cmo-rapport_wargaming-.pdf.

Julian Stafford Corbett. *Some Principles of Maritime Strategy, by Julian Stafford Corbett*. 1911. URL https://www.gutenberg.org/files/15076/15076-h/15076-h.htm.

P.P. Perla. *The Art of Wargaming: A Guide for Professionals and Hobbyists*. Twayne's Masterworks Studies. Naval Institute Press, 1990. ISBN 978-0-87021-050-1. URL https://books.google.dk/books?id=C_sjPhgaEzwC.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, May 1992. ISSN 0885-6125, 1573-0565. doi: 10.1007/BF00992698. URL http://link.springer.com/10.1007/BF00992698.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction - Free Computer, Programming, Mathematics, Technical Books, Lecture Notes and Tutorials*. MIT Press, Cambridge, MA, 2018; eBook, 2nd edition, 2018. ISBN 978-0-262-03924-6. URL https://mitpress.ublish.com/ebook/reinforcement-learning-an-introduction-2-preview/2351/Cover.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with Deep Reinforcement Learning, 2013. URL https://arxiv.org/abs/1312.5602. Version Number: 1.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, May 1992. ISSN 0885-6125, 1573-0565. doi: 10.1007/BF00992696. URL http://link.springer.com/10.1007/BF00992696.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017. URL https://arxiv.org/abs/1707.06347. Version Number: 2.

Vijay Konda and John Tsitsiklis. Actor-Critic Algorithms. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. 2016. doi: 10.48550/ARXIV.1602.01783. URL https://arxiv.org/abs/1602.01783. Publisher: arXiv Version Number: 2.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning, 2015. URL https://arxiv.org/abs/1509.02971. Version Number: 6.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, 2018. URL https://arxiv.org/abs/1801.01290. Version Number: 2.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments, 2017. URL https://arxiv.org/abs/1706.02275. Version Number: 4.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning, 2018. URL https://arxiv.org/abs/1803.11485. Version Number: 2.

Philip M. Morse. *Methods Of Operations Research*. Chapman And Hall Limited., 1951. URL http://archive.org/details/methodsofoperati030158mbp.

Ömer Faruk Arar and Kürşat Ayan. A flexible rule-based framework for pilot performance analysis in air combat simulation systems. *TURKISH JOURNAL OF ELECTRICAL ENGINEERING & COMPUTER SCIENCES*, 21: 2397–2415, 2013. ISSN 13000632, 13036203. doi: 10.3906/elk-1201-50. URL https://journals.tubitak.gov.tr/elektrik/vol21/iss8/21.

Gary J. Briggs. Harnessing Constructive Simulations for Reinforcement Learning. Technical report, RAND Corporation, August 2024. URL https://www.rand.org/pubs/research_reports/RRA1722-6.html.

Kyle Bunch, Alexander C. Hou, Ryan Haberman, Marissa Herron, Anthony Jacques, and Gary J. Briggs. Risk Assessment of Reinforcement Learning AI Systems: Looking Beyond the Technology. Technical report, RAND Corporation, July 2024. URL https://www.rand.org/pubs/research_reports/RRA1473-1.html.

Claude E. Shannon. XXII. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, March 1950. ISSN 1941-5982, 1941-5990. doi: 10.1080/14786445008521796. URL http://www.tandfonline.com/doi/abs/10.1080/14786445008521796.

R. Landauer. Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development*, 5(3):183–191, July 1961. ISSN 0018-8646, 0018-8646. doi: 10.1147/rd.53.0183. URL http://ieeexplore.ieee.org/document/5392446/.

E Zermelo. Uber eine Anwendung der Mengenlehre auf die Theorie des Schachspiels. In *Proceedings of the Fifth International Congress of Mathematicians (Cambridge 1912), Vol. 2*, pages 501–504. Cambridge University Press, Cambridge, 1913.

Antti Valmari. The state explosion problem. In Gerhard Goos, Juris Hartmanis, Jan Leeuwen, Wolfgang Reisig, and Grzegorz Rozenberg, editors, *Lectures on Petri Nets I: Basic Models*, volume 1491, pages 429–528. Springer Berlin Heidelberg, Berlin, Heidelberg, 1998. ISBN 978-3-540-65306-6 978-3-540-49442-3. doi: 10.1007/3-540-65306-6_21. URL http://link.springer.com/10.1007/3-540-65306-6_21. Series Title: Lecture Notes in Computer Science.

Mark M. Lowenthal. Carl Von Clausewitz. *On War* . Edited and Translated by Michael Howard and Peter Paret. Commentary by Bernard Brodie. Princeton: Princeton University Press. 1976. Pp. xii, 717. $18.50. *The American Historical Review*, 82(3):608–609, June 1977. ISSN 1937-5239, 0002-8762. doi: 10.1086/ahr/82.3.608-a. URL https://academic.oup.com/ahr/article/82/3/608/80335.

Isaac Newton. *The Principia: The Authoritative Translation and Guide: Mathematical Principles of Natural Philosophy*. University of California Press, February 2016. ISBN 978-0-520-96481-5. doi: 10.1525/9780520964815. URL https://www.degruyter.com/document/doi/10.1525/9780520964815/html.

J E Lennard-Jones. Cohesion. *Proceedings of the Physical Society*, 43(5): 461–482, September 1931. ISSN 0959-5309. doi: 10.1088/0959-5309/43/5/301. URL https://iopscience.iop.org/article/10.1088/0959-5309/43/5/301.

David J. Griffiths. *Introduction to Electrodynamics*. Cambridge University Press, 4 edition, June 2017. ISBN 978-1-108-33351-1 978-1-108-42041-9. doi: 10.1017/9781108333511. URL https://www.cambridge.org/highereducation/product/9781108333511/book.

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing Function Approximation Error in Actor-Critic Methods, 2018. URL https://arxiv.org/abs/1802.09477. Version Number: 3.

Twin Delayed DDPG — Spinning Up documentation. URL https://spinningup.openai.com/en/latest/algorithms/td3.html.

Fredrick Sakala. Three Levels of War. *Air and Space Power Mentoring Guide*, Vol 1, 1997.

Appendix A: Machine Learning and Neural Networks

Machine Learning is a category of computational methods that allows systems to improve task solving performance by learning from experience. Broadly speaking machine learning is divided into categories, unsupervised learning, supervised learning and reinforcement learning. They are implemented in different areas of problemsolving depending on their use-case.

A.1 Unsupervised Learning

works on unlabeled data - i.e data without a truth or ground value - and it works by identifying hidden patterns or embedded structures in the data sets. Formally

$$D = \{x_i\}_{i=1}^{N}, \quad \text{where } x_i \in \mathbb{R}^d$$

Where d is the dimensionality of the data. The applicability of unsupervised learning include dimensionality reduction of data, visualization with various nonlinear projections onto 2 dimensions like with the UMAP or t-SNE algorithms or grouping clusters within the data with k-means.

If UMAP ends up being used, include explanation here!

A.2 Supervised Learning

in trained on labeled data, where the truth or ground value is known, and through training on labeled data is able to make predictions about unlabeled data. Formally

$$D = \{(x_i, y_i)\}_{i_1}^{N}, \quad \text{where } x_i \in \mathbb{R}^d, y_i \in \mathbb{R}^d$$

where the goal is to train a model which act as a function on labeled data such that $f : \mathbb{R}^d \to \mathbb{R}$ that minimizes prediction errors on new unseen unlabeled data, by convention input and output data is denoted with a hat if the model has not been trained on the data $f(\hat{x}_i) = \hat{y}_i$.

There are two classes of problems that supervised learning are able to solve, *Regression* and *Classification*. In regression tasks, the data is labeled with a continuous vector. Typical problems could be the prediction of a resultant temperature based on heating configurations by a model trained on historic indoor thermometer readings, with known heating configurations. In classification, the data is labeled with a discrete vector, representing grouping - the goal of the model is then to accurate predict which group new data belongs to. An example could be classification of mails into spam or not-mail classes, using the content of the email as input[1].

A wide range of methods exists with are able to solve supervised learning problems such as boosted decision trees and neural networks.

[1] Using natural language processing in order to quantify the textual content

A.3 The Perceptron

One of the fundamental building blocks of a simple supervised learning model is the *Perceptron* classifier. It was among the first to be developed and is the foundation behind neural networks, it takes an input and defines a linear decision boundary for a binary output and in that way classifies linearly the input into two classes - mathematically expressed as;

$$y = f(x) = \begin{cases} 1 & \text{if } w * x + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The linear decision boundary, act as a activation function separating the data into two categories. The parameters $w \in \mathbb{R}^d$ represents the model's weights and $b \in \mathbb{R}$ is the bias term. At first, the perceptron is initialized with a initial set of weight and bias values (random, median or just good guesses), and after each iteration, it updates the weights and bias according to;

$$w_{old} + \mu(y - \hat{y})x \rightarrow w_{new}, \quad b_{old} + \eta(y - \hat{y}) \rightarrow b_{new}$$

Where η is the *learning rate*, a hyperparameter that adjusts the weights and bias of the model.

A.4 Neural Networks

A neural network significantly extends the utility of the perceptron by arranging them in sequence in multiple layers and introduces nonlinearities with different activation functions, enabling the model to capture delicate details of complex data relationships. Neural networks consists of an input layer, that receives data $\hat{x}_i$ from a dataset and an output layer that produces the final prediction $\hat{y}_i$. In between there may be many intermediate hidden layers, which perform nonlinear transformations through learnable weight, biases and activation functions.

Mathematically, each neural network layer perform the following computation;

$$\vec{z}_{(l)} = \mathbf{W}_{(l)}\vec{a}_{(l-1)} + \vec{b}_{(l)}, \quad \vec{a}_{(l+1)} = \sigma(z_{(l)})$$

where l is the layer index, $\vec{a}$ is the output of the previous layer and thus the input of the current layer, $\vec{z}$ is the output of the layer onto the next, $\mathbf{W}$ is a weight matrix, $\vec{b}$ is a bias vector and finaly σ is an activation function. The data propagates all through all the layers receiving a nonlinear transformation and produces an final output $\hat{y}_i$.

a significant part of the utility of neural network model is the nonlinearity introduced from various activation functions. In the perceptron the activation function is linear, but computer scientist have discovered many other usefull functions. Popular ones include *Rectified linear unit* also known as *ReLU*,

Leaky ReLU, *Hyperbolic Tangent* and *Sigmoid*, all with different properties and use-cases.

The prediction error is then measured with a loss function $\mathcal{L}$, which can take different forms, frequently used in regression problems is the mean squared error[2]:

$$\mathcal{L}(y, \hat{y}_i) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

The learning process for the neural network then becomes an optimization problem, where the loss has to be minimized.

A.5 Backpropagation

This is done with the backpropagation algorithm, which uses gradient descent to minimize the loss function by changing the weight matrix and bias vectors of each layer. Gradients are then computed layer by layer with the chain rule, recursively backwards through the network;

$$\frac{\partial L}{\partial \mathbf{W}_{(l)}} = \frac{\partial L}{\partial \vec{a}_{(l)}} \frac{\partial \vec{a}_{(l)}}{\partial \vec{z}_{(l)}} \frac{\partial \vec{z}_{(l)}}{\partial \mathbf{W}_{(l)}}$$

The parameters are then updated with gradient descent with learning rate η

$$\mathbf{W}_{(l)} - \eta \frac{\partial L}{\partial \mathbf{W}} \rightarrow \mathbf{W}_{(l),new} \ , \quad \vec{b}_{(l)} - \eta \frac{\partial L}{\partial \vec{b}_{(l)}} \rightarrow \vec{b}_{(l),new}$$

By continuously iterating, the algorithm approximates the complex mapping between input data and truth value, and can apply it to new unseen data. By slowly lowering the learning rate, it allows the algorithm to "zoom-in" on the right solutions.

[2]Whereas cross-entropy loss is often seen in classification